



Data Warehouses

Lecture 2

Lecture Goals

Agenda:

- Quick knowledge stance check

 - Relational algebra, Execution plans*

- Quick intro to OLAP/OLTP

- Querying data using SQL – data analysis perspective

 - Introduction, Pivot and Case, Grouping sets*

Relational Algebra

Five primitive operators of Codd's algebra are

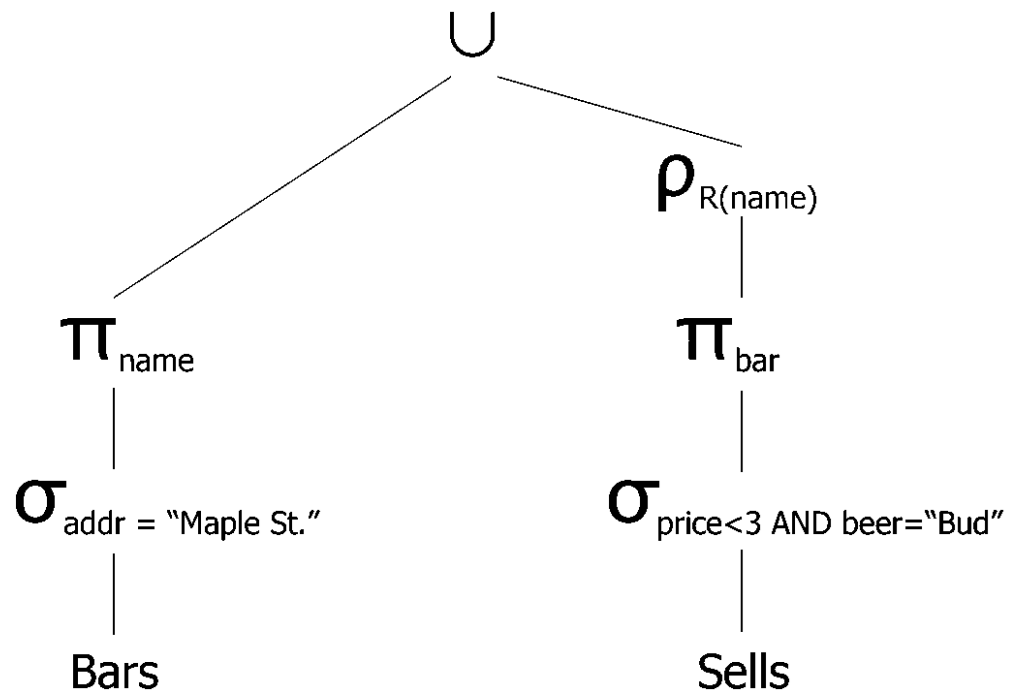
selection, projection, Cartesian product (also called the cross product or cross join), set union, and set difference.

We also have

intersection, other products and joins, renaming, aggregates, etc.

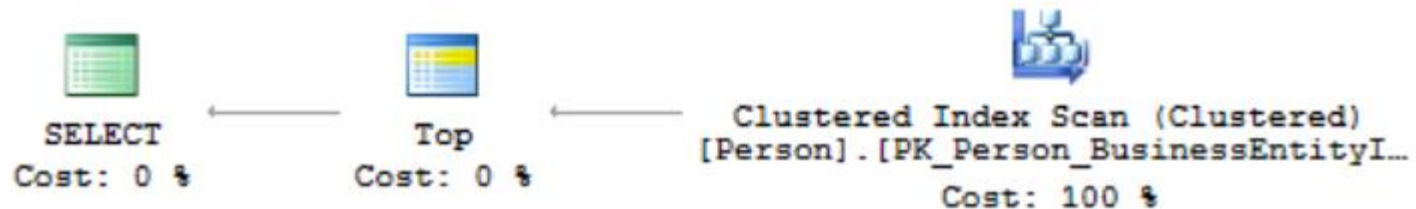


As a Tree:



Reading plans

```
SELECT TOP(1) Title, FirstName, LastName  
FROM Person.Person;
```



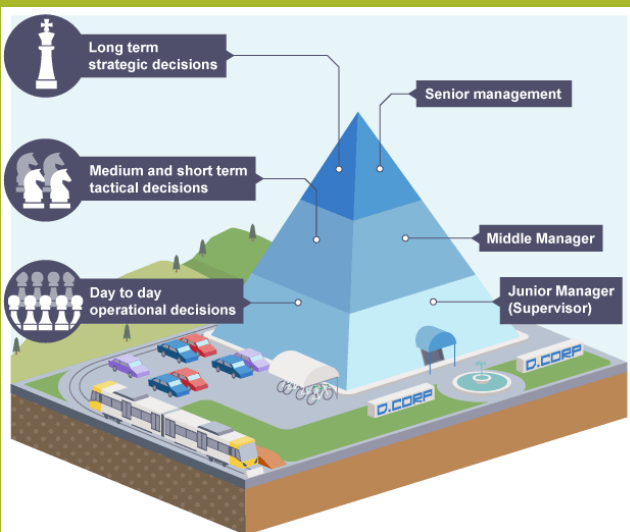


Needs for data have evolved around two main purposes

Data capture

Data analysis

Operational vs Strategic



Main two layers in the company:

Executive - **Operational** Decisions

Operational activities are data-generating activities made in real time on a daily or weekly basis, so their scope is narrow.

Operational processes require data to function effectively

such as vehicle loading or dispatching, shipment, and warehouse routines

based on lots of detailed data

Decision-making - **Strategic** Decisions

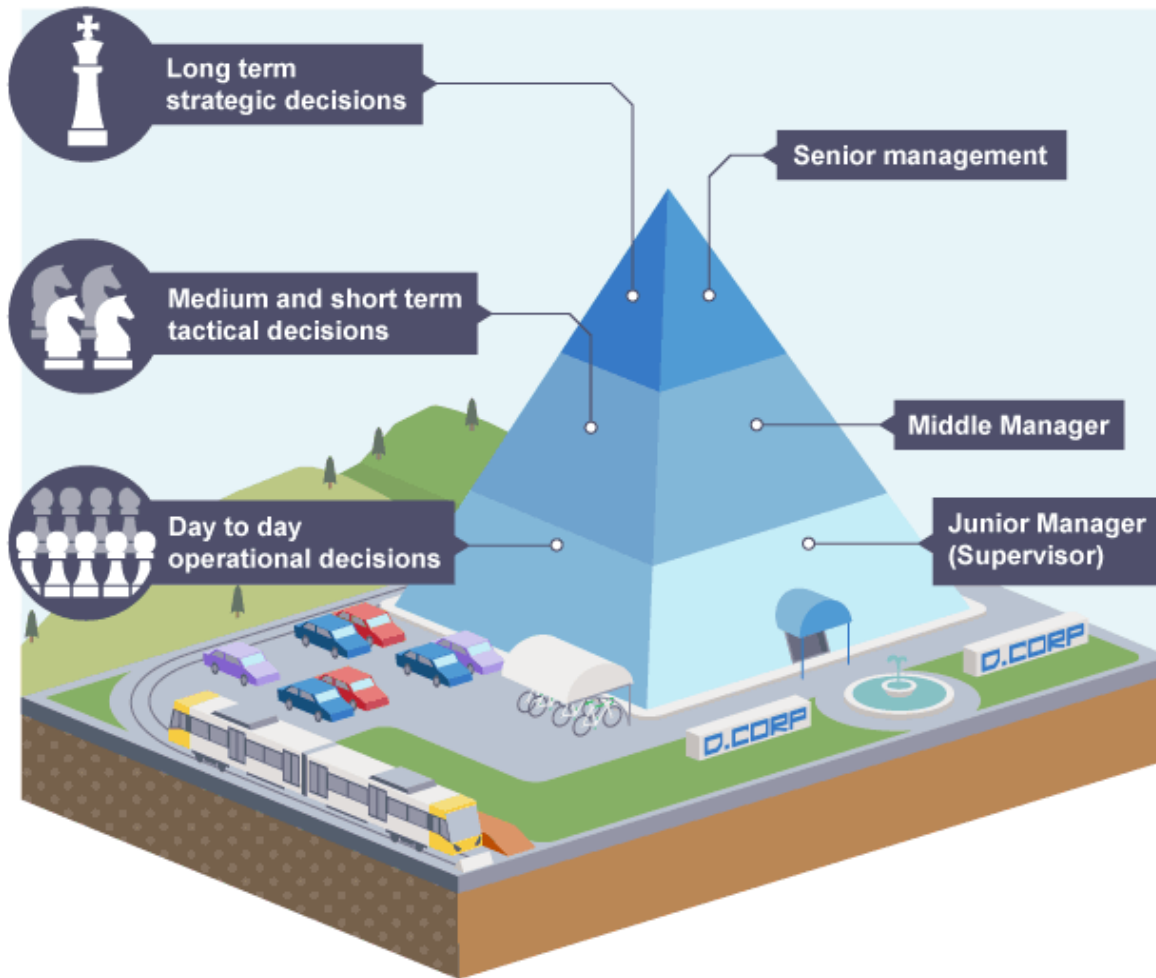
business objectives and mission statements, as well as marketing and customer-service strategies.

long-term kinds of decisions made over one or more years.

operational data is the raw material for insights.

made by executive administrators, top managers, and stockholders.

such as capital reduction (the level of investment, which depends on owned equipment and inventories), cost reduction (the total cost of transportation and storage), service-level improvement (customer satisfaction and order cycle time)



Operational vs Strategic

Two main layers in the company:

Executive

automation of the company's day-to-day operations

Decision-making

providing information to the management.

How
organisational
needs for data
has evolved
around two
main
purposes.

Data Capture

it all started with the sole objective of capturing transactional data for operational records

customers' contact details, sales orders, invoices, payments, product details and so on.

databases were created to serve 2 purposes:

Record and keep track of business activities quickly and digitally

(so that we don't have to deal with papers flying everywhere and getting lost forever)

Help business users to complete their day-to-day tasks more effectively

(e.g. whether an order has been shipped or how much inventory we are having)

Simply put, focusing on the present and making sure business transactions happen as smoothly as possible

Example

Database - working in a supermarket

What are some typical operations?

What is required of the database?

How is its performance measured?

How is the data organized/structured?

What about ability to analyse data?

What about the ability to create reports?



Database

What is data?

Where does the data come from?

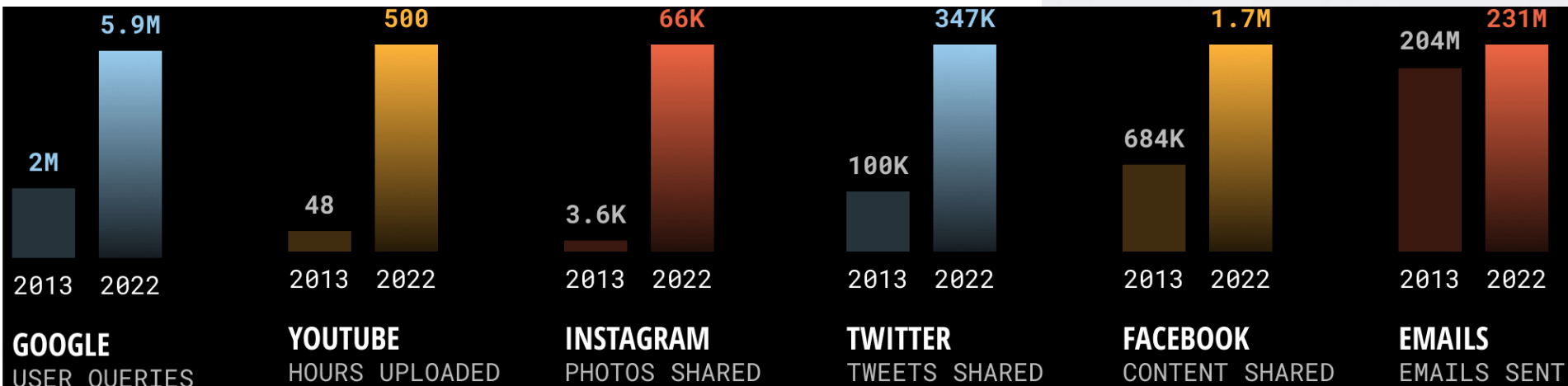
What is a database?

Why do we need databases ...

How do we use databases ...

How do we develop databases ...

What are the main components of a database system?



Database

Oracle:

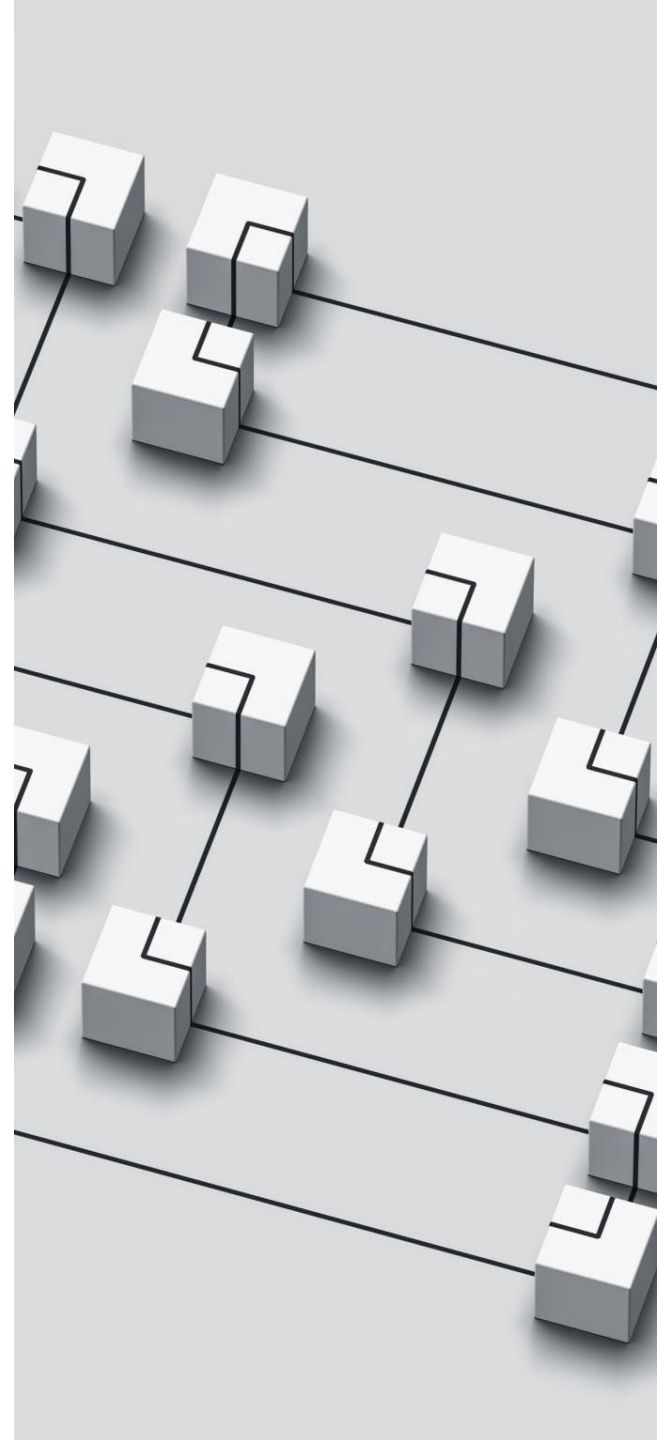
A database is an organized collection of structured information, or data, typically stored electronically in a computer system.

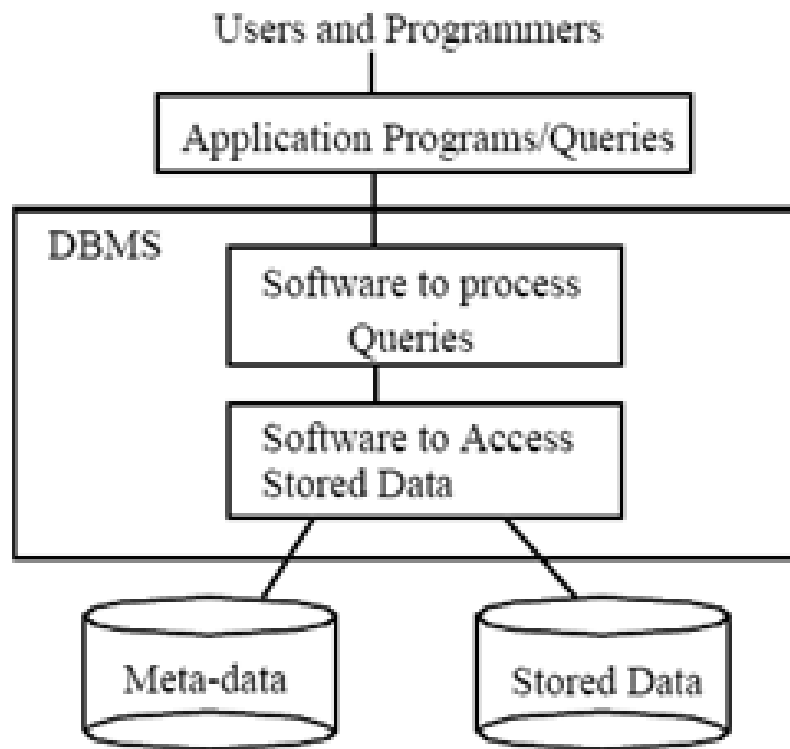
A database is usually controlled by a database management system (DBMS).

Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system, often shortened to just database.

Connolly and Begg, 2014:

Database management system (DBMS) is a "software system that enables users to define, create, maintain and control access to the database".





Database System

Database

Database management system

Database engine

SQL Server Database

The diagram illustrates the architecture of the SQL Server

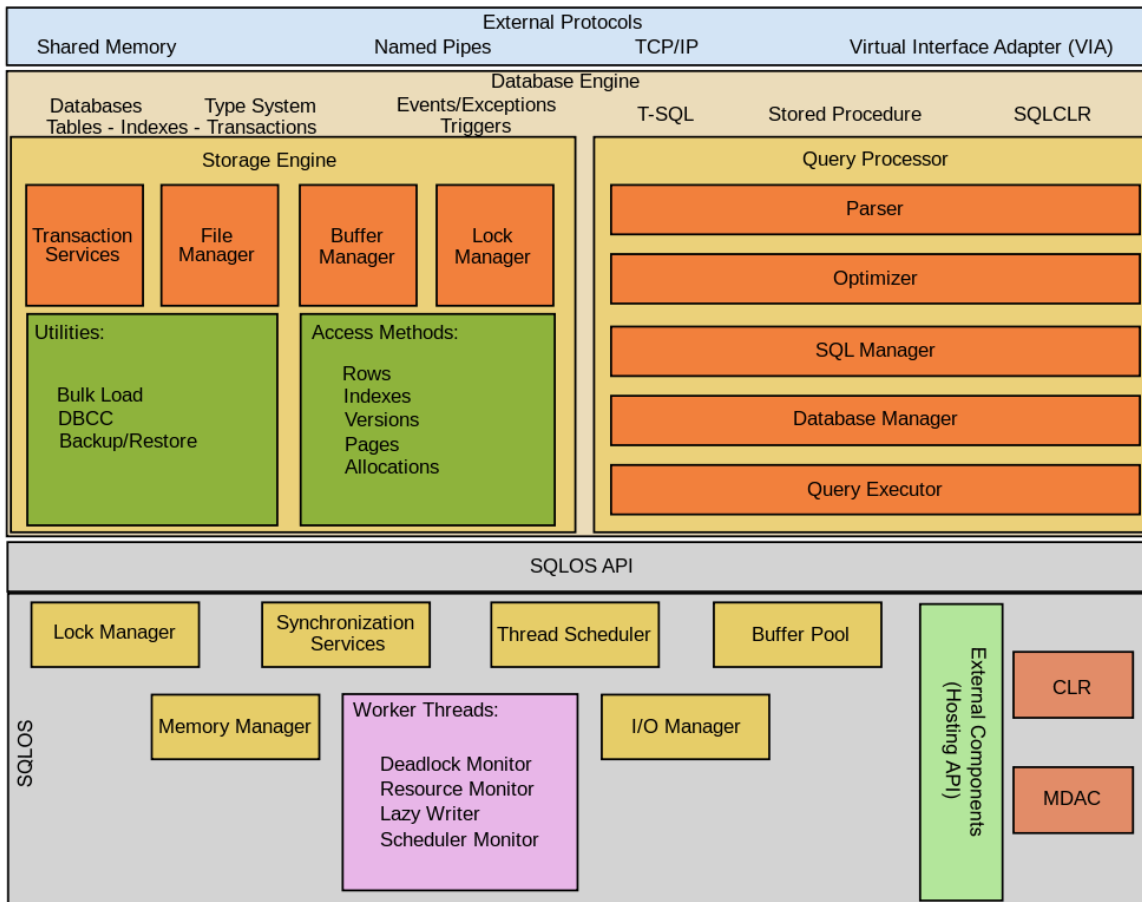
Consists of two main components:

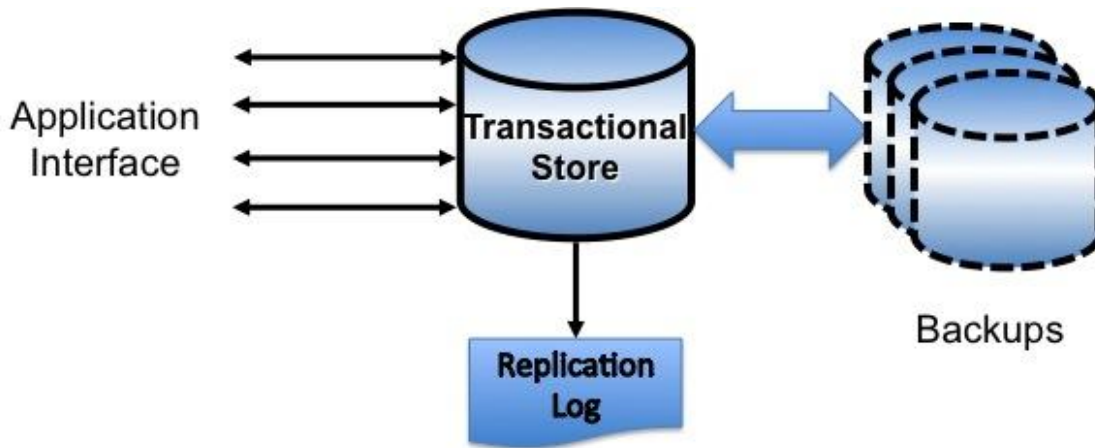
Database Engine

Storage Engine

Relational Engine

SQLOS





Database System

Once again:

why do we need databases
store, retrieve, update, delete data

what are the primary usage scenarios for databases

*handling commercial transaction**

ACID

key properties to ensure consistent, safe and robust database modification when saved.

How
organisational
needs for data
has evolved
around two
main
purposes.

Data Analysis

Businesses quickly realised living in the moment wasn't good enough.

New questions arose.

How are we performing as compared to last month?

Why was there a fall in revenue in the last two quarters?

What have our customers regularly complained about since the beginning of the year?

The need to capture all operational records is still indispensable

now another equally important objective: analyse historical data for decision-making.

database – once built to capture transactional data – wasn't suitable to empower management with enough facts and historical insights for making informed decisions.

Vast difference between capturing present records and storing historical data across a variety of data sources for analysis

we need another tool.



Information systems

“Through measurement comes knowledge.”

Heike Kamerlingh Onnes

Information system vs business intelligence system

Business Intelligence (BI) systems are a specific category of Information System that are purpose-built for analytical processing.

Analogy think of a hospital.

Information Systems in a Hospital, includes:

Patient Record System (Electronic Health Records - EHR)

Billing System

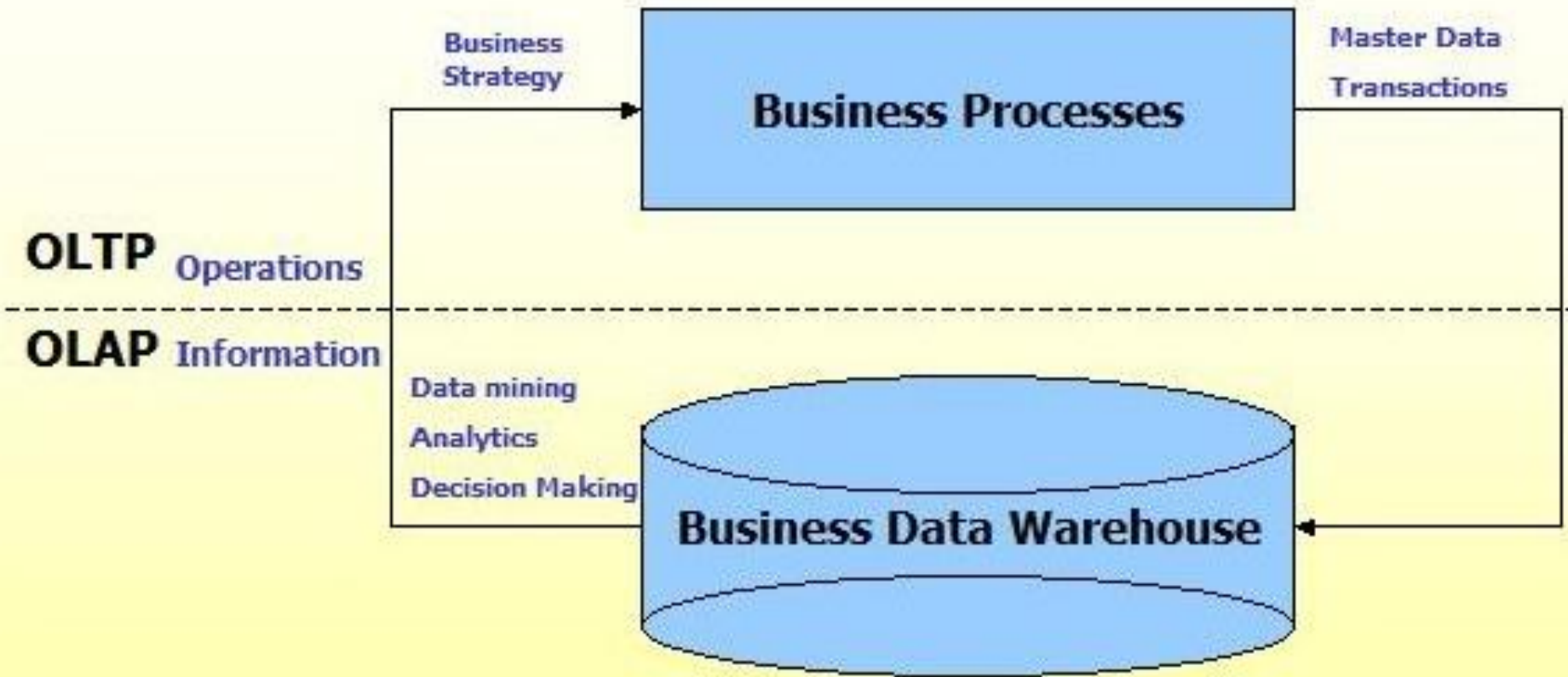
Pharmacy System

Business Intelligence (BI) System for Hospital focuses on analytical IS that takes data from EHR, billing, pharmacy, etc. to analyze:

Patient demographics and disease trends (strategic planning for resource allocation).

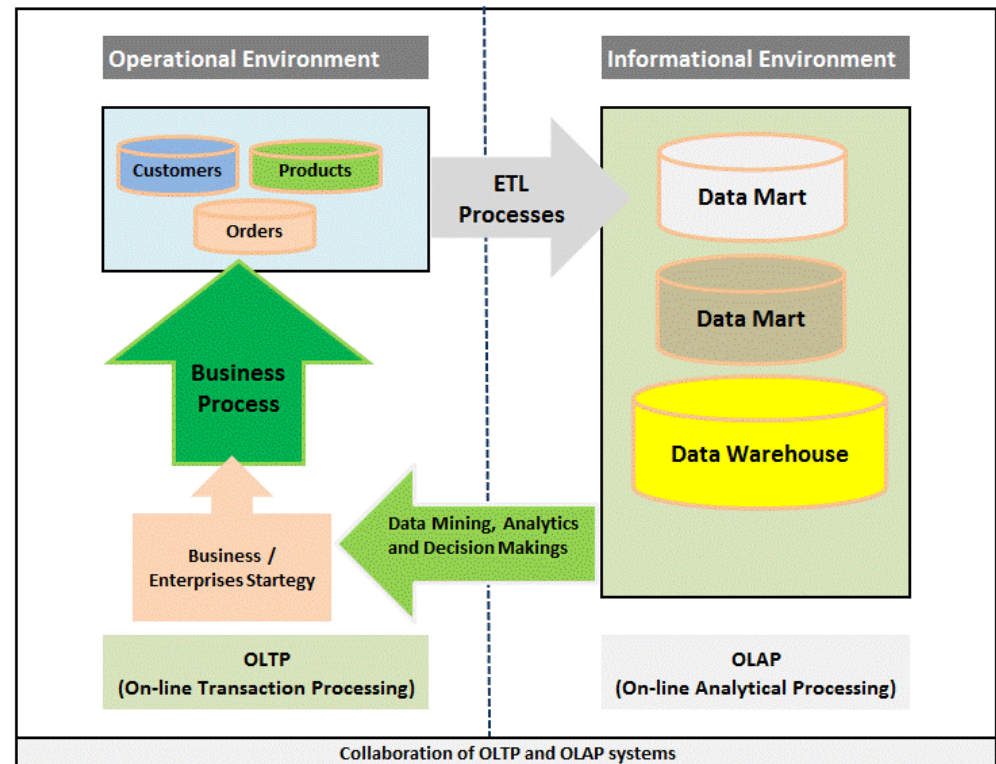
Hospital performance metrics (KPI dashboards for management).

A data warehouse is an information delivery system.



Operational vs Strategic

Operational & Strategic



OLTP / OLAP

There are two types of data processing systems:

- online analytical processing (OLAP)

- online transaction processing (OLTP)

The question isn't which to choose, but how to make the best use of both processing types for your situation.

OLAP

The term OLAP was first used in 1993

E. F. Codd (who defined the rules for relational data model in 1985) delivered a report:

Codd E.F., Codd S.B., and Salley C.T. "Providing OLAP (On-line Analytical Processing) to User-Analysts: An IT Mandate". Codd & Date, Inc 1993.

Codd indicated twelve features that should be present in any OLAP application

four points are probably the most significant:

Multidimensional conceptual view.

Transparency.

Flexible reporting.

Unlimited dimensional and aggregation levels.



Example

Consider three systems in an org.

Accounting and finances

invoices

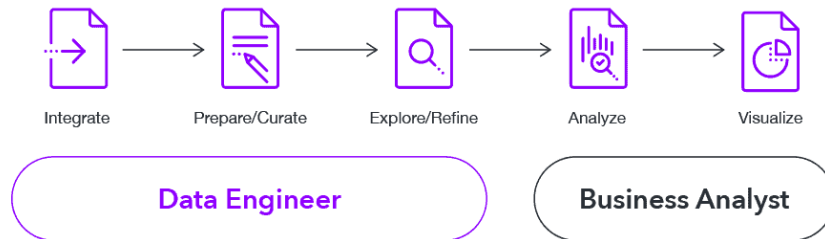
Production

production costs

CRM

Marketing costs

Creating a report that puts together inflows and outflows for a given customer



OLTP / OLAP data

Regardless of whether all data is available

the process of meeting user needs requires a lot of work before the data becomes useful

Interesting analogy:

You wouldn't want to eat a cup of flour, a cube of raw egg butter and a few tablespoons of sugar separately

You would rather prefer a baked cake

*only after proper preparation process
e.g. the right amount of time at the right temperature, turns all ingredients into something delicious*

Raw data is "tasteless" to a manager

it is difficult to make decisions based on them

they are often inconsistent, incomplete, unformatted and full of dirt

It is necessary to properly prepare raw data before their consumption

related to aspects such as: integration, design, modeling, architecture, etc.

OLTP database

Operational databases

basic access pattern

processing business transactions:

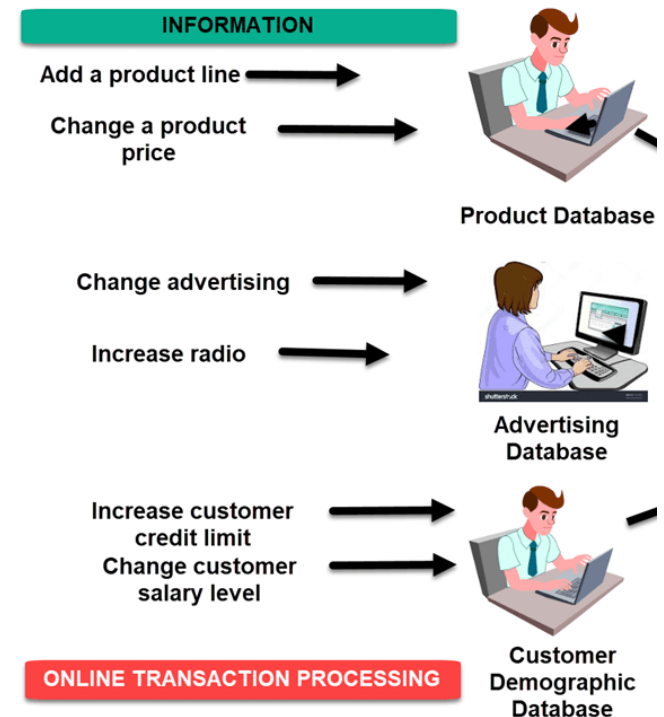
app looks up some small number of records from the database

records are inserted or updated based on user's input

Interactive

have an architecture that supports efficient inserts, updates, and deletes, usually using index seek operations as the data access method

Manage Employees					
				Delete	Add New Employee
☐	Name	Email	Address	Phone	Actions
☐	Thomas Hardy	thomashardy@mail.com	89 Chiaroscuro Rd, Portland, USA	(171) 555-2222	Edit Delete
☐	Dominique Perrier	dominiqueperrier@mail.com	Obere Str. 57, Berlin, Germany	(313) 555-5735	Edit Delete
☐	Maria Anders	mariaanders@mail.com	25, rue Lauriston, Paris, France	(503) 555-9931	Edit Delete
☐	Fran Wilson	franwilson@mail.com	C/ Araquil, 67, Madrid, Spain	(204) 619-5731	Edit Delete
☐	Martin Blank	martinblank@mail.com	Via Monte Bianco 34, Turin, Italy	(480) 631-2097	Edit Delete
Showing 5 out of 25 entries					
				Previous	1 2 3 4 5 Next



OLAP database

Databases started being used for data analytics

different access patterns

scan over a huge number of records

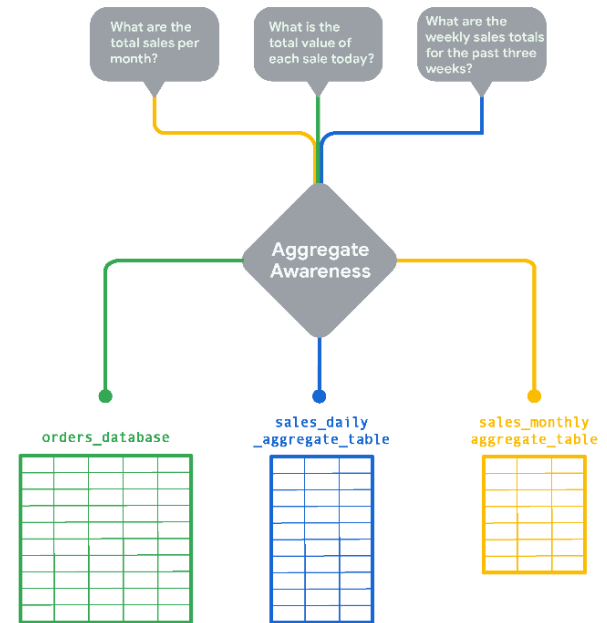
typically reading only few attributes/columns per record

calculating aggregate statistics

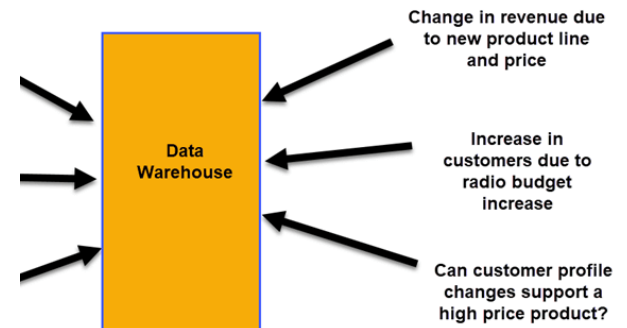
counting, summing, averaging

Interactive

architecture to support analytical workloads, which are primarily reads using table scans

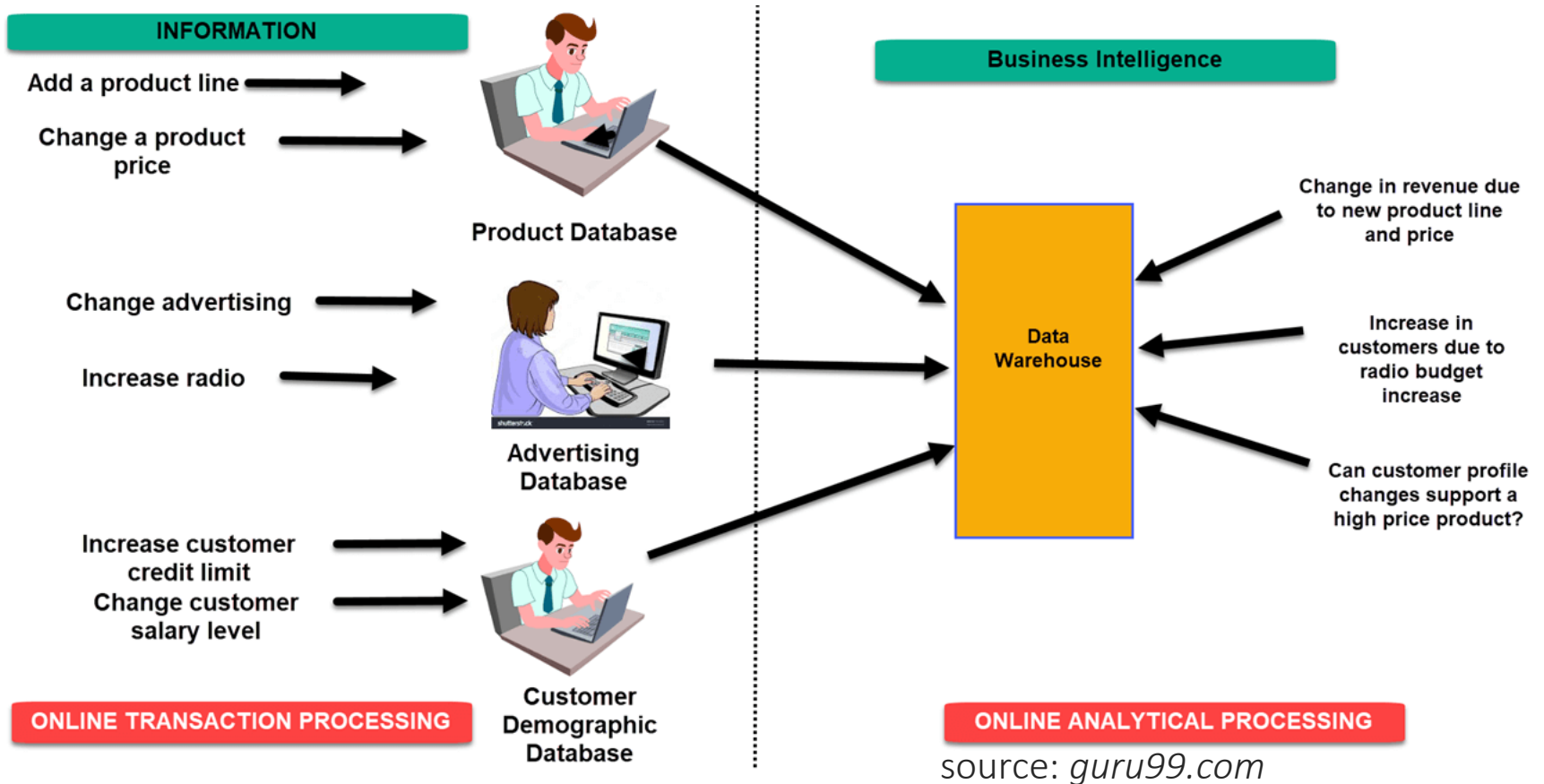


Business Intelligence



ONLINE ANALYTICAL PROCESSING

OLTP / OLAP



OLTP / OLAP

Information driven by online transaction processing

Product Database

- Add a Product Line
- Change a Product Price

Advertising Database

- Change Advertising Timetable
- Increase Radio Advertising Budget

Consumer Demographic Database

- Increase Customer Credit Limit
- Change Salary Level



DATA WAREHOUSE

Business intelligence driven by analytical processing

How many products sold due to TV ads last month?

If inventory levels drop by 10%; will customer shop elsewhere?

Which customer demographic is performing best for product A?

Operational VS Analytical Reporting

Feature	Operational Reporting	Analytical Reporting
<i>Purpose</i>	Monitor daily operations, immediate action	Understand trends, strategic decisions
<i>Focus</i>	Current status, immediate performance	Trends, patterns, insights over time
<i>Data Scope</i>	Transactional, granular, real-time/near real-time	Aggregated, historical, summarized
<i>Data Source</i>	OLTP systems	Data Warehouse, OLAP systems
<i>Frequency</i>	Frequent (daily, hourly, real-time)	Less frequent (weekly, monthly, ad-hoc)
<i>Users</i>	Operational staff, supervisors, managers	Managers, executives, analysts, strategists
<i>Questions</i>	"What is happening now?", "Is it working?"	"Why is this happening?", "What can we improve?"
<i>Report Structure</i>	Standardized, pre-defined, structured	Flexible, exploratory, often interactive
<i>Data Granularity</i>	Detailed, granular	Summarized, aggregated
<i>Orientation</i>	Action-oriented, exception-based	Insight-oriented, trend-focused
<i>Visualization</i>	Tables, simple charts	Charts, graphs, dashboards, visualizations

OLTP Database Design

ERD

Abstract data model

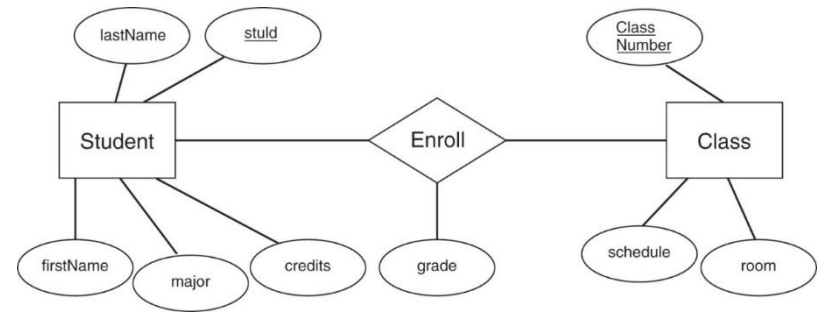
Utilises

Entities

Relations

Focused on

Operational usage



UML data model

Abstract data model

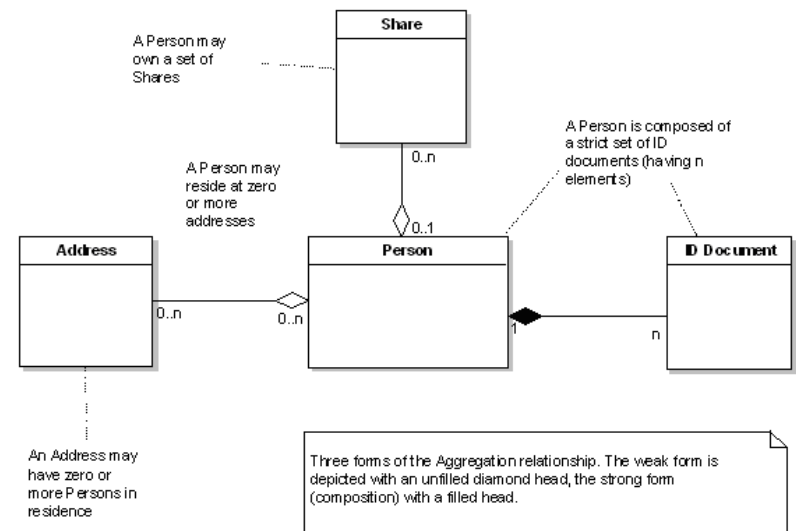
Utilises

Classes

Associations

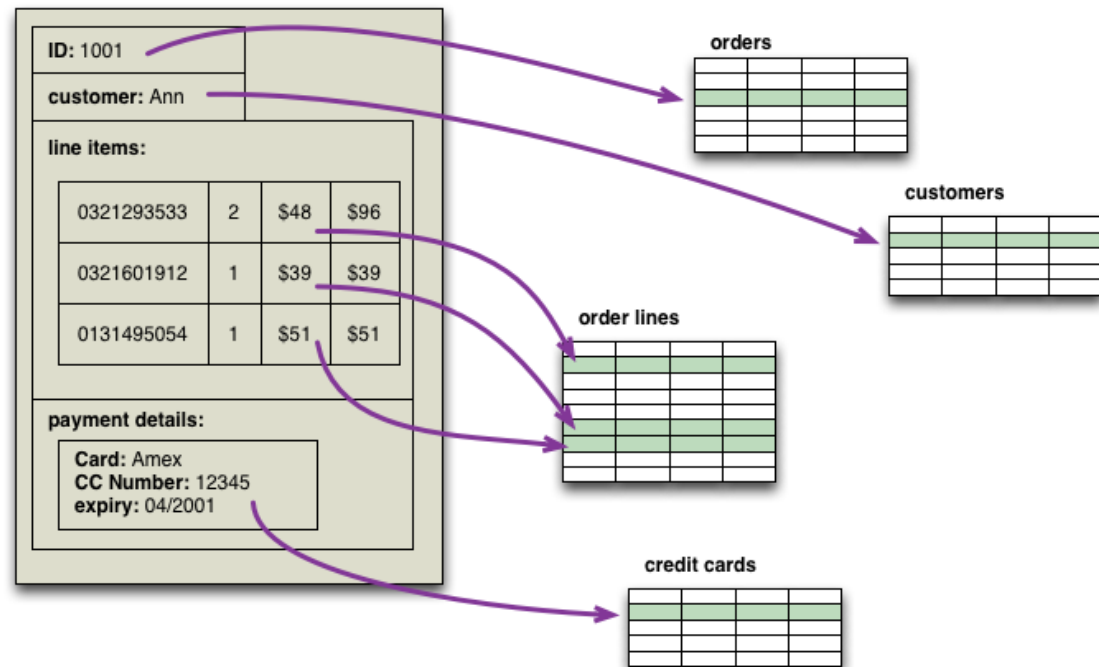
Focused on

Operational usage



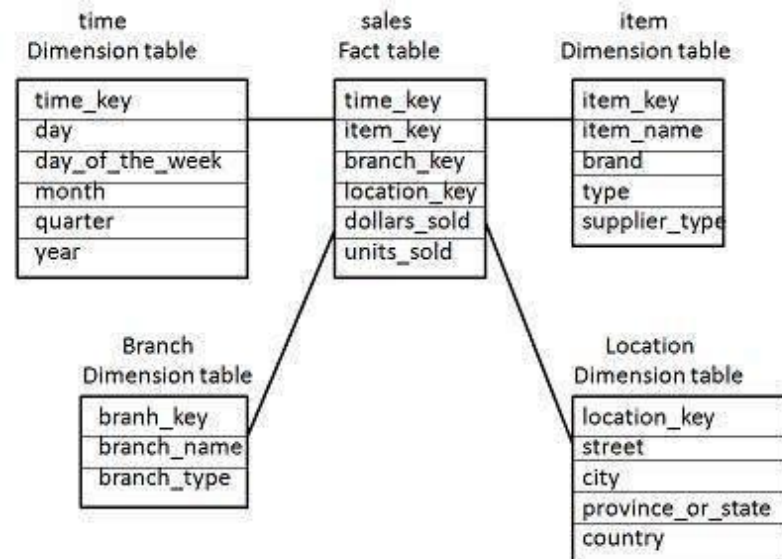
OLTP Database Design

Database
support clerical
operations



OLAP Database Design

Analytical needs
Oriented towards
Facts
Dimensions
Focused on
Analytical
processing
Requires different
design approach



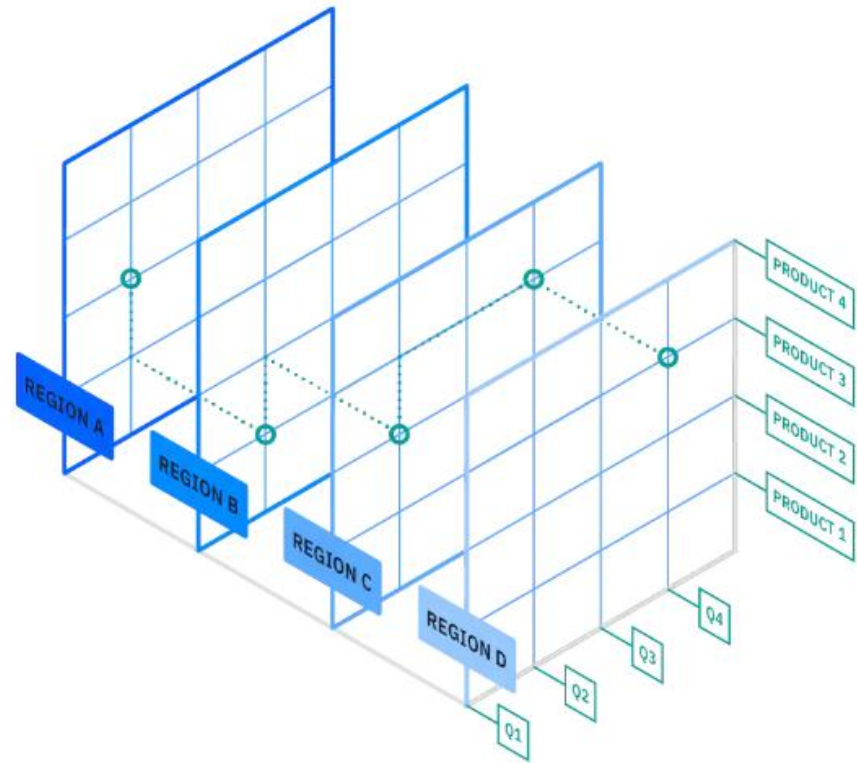
OLAP Database Design

Data warehouse
support data analysis

“There are typically a number of different dimensions from which a given pool of data can be analysed.”

“This plural perspective, or Multidimensional Conceptual View appears to be the way most business persons naturally view their enterprise.”

E.F. Codd, S.B. Codd and C.T. Salley, 1993



OLTP/OLAP - Handling change

Handling change

Imagine that James has changed his location:

From New York

To Chicago

How to handle this change?

ID	Year	Name	City
1	2000	James	New York

OLTP/OLAP - Handling change

In OLTP

Rather straightforward

Before

ID	Year	Name	City
1	2000	James	New York

After

ID	Year	Name	City
1	2004	James	Chicago

OLTP/OLAP - Handling change

Before

Table 1 - Before Change

ID	Year	Name	City
1	2000	James	New York

In OLAP

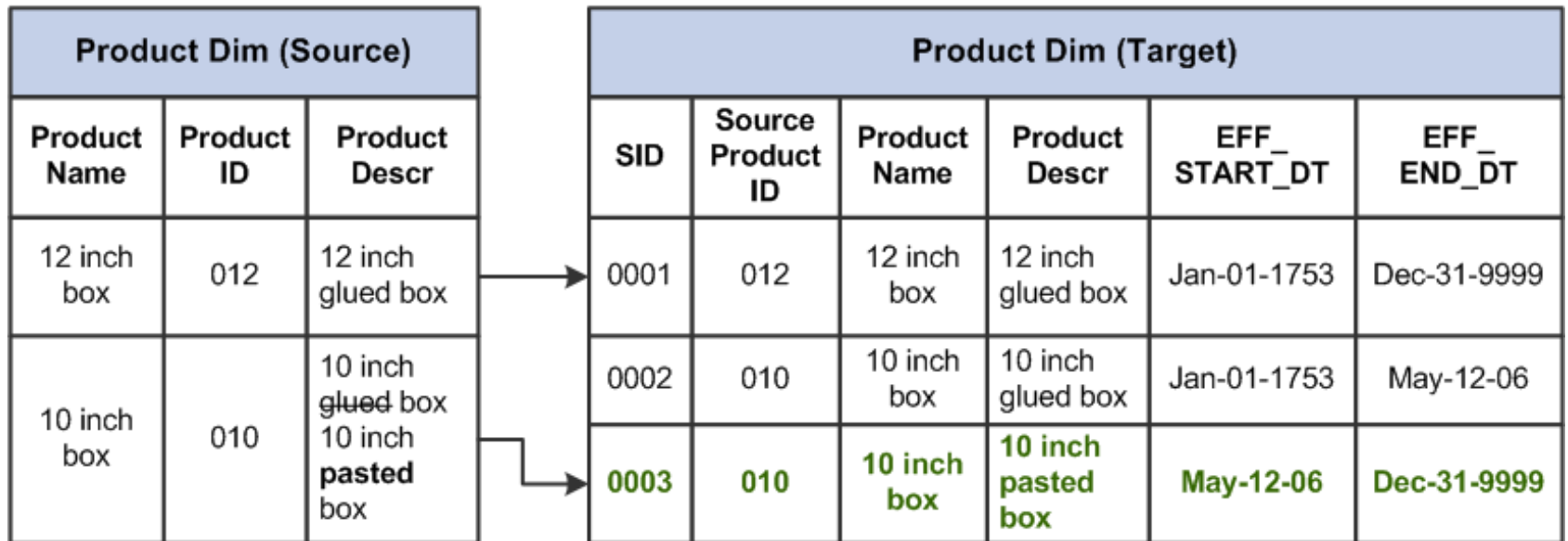
Sometimes a more complex process is required

After

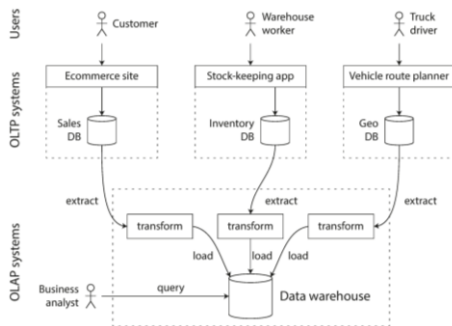
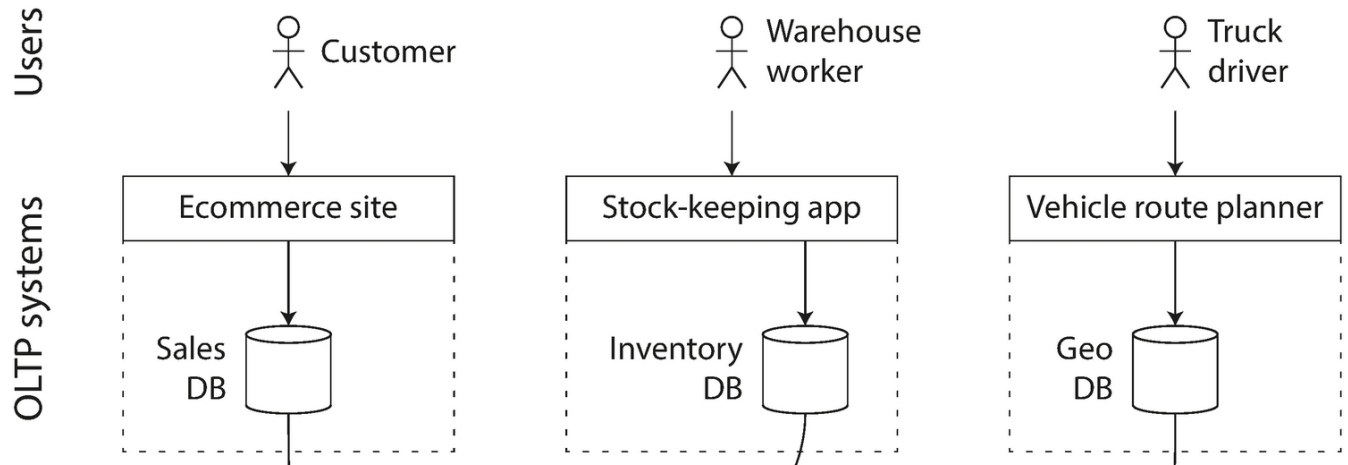
Table 2 - After Change

ID	Year	Name	City
1	2000	James	New York
1	2004	James	Chicago

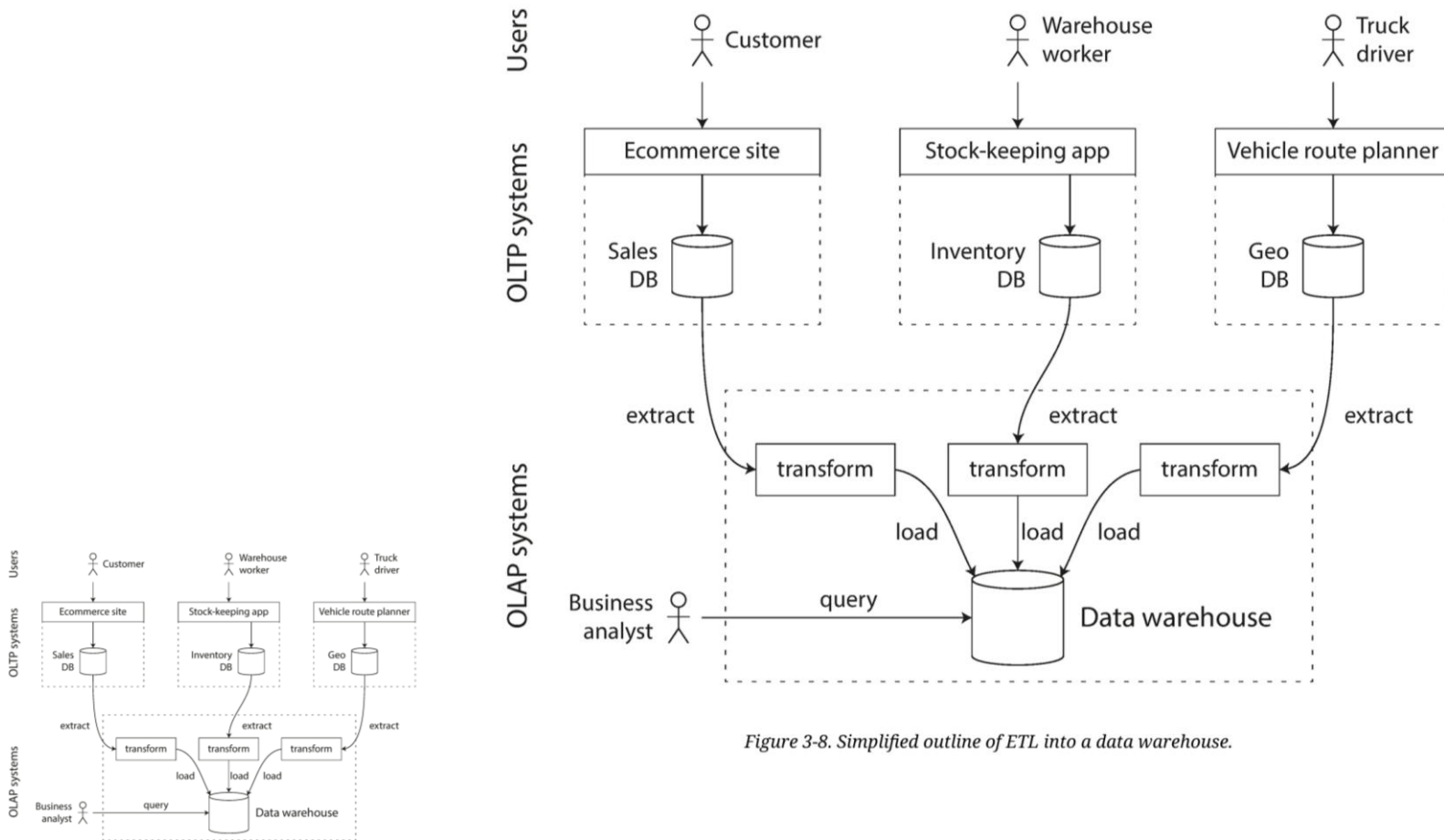
OLTP/OLAP – Handling data



Operational – Strategic



Operational – Strategic



OLTP

OLTP

on-line transaction processing (OLTP)

Goal

supporting in the day-to-day operations

Operational usage of data

Main tasks

processing of a huge number of concurrent transactions

insuring data integrity

Transactional DB

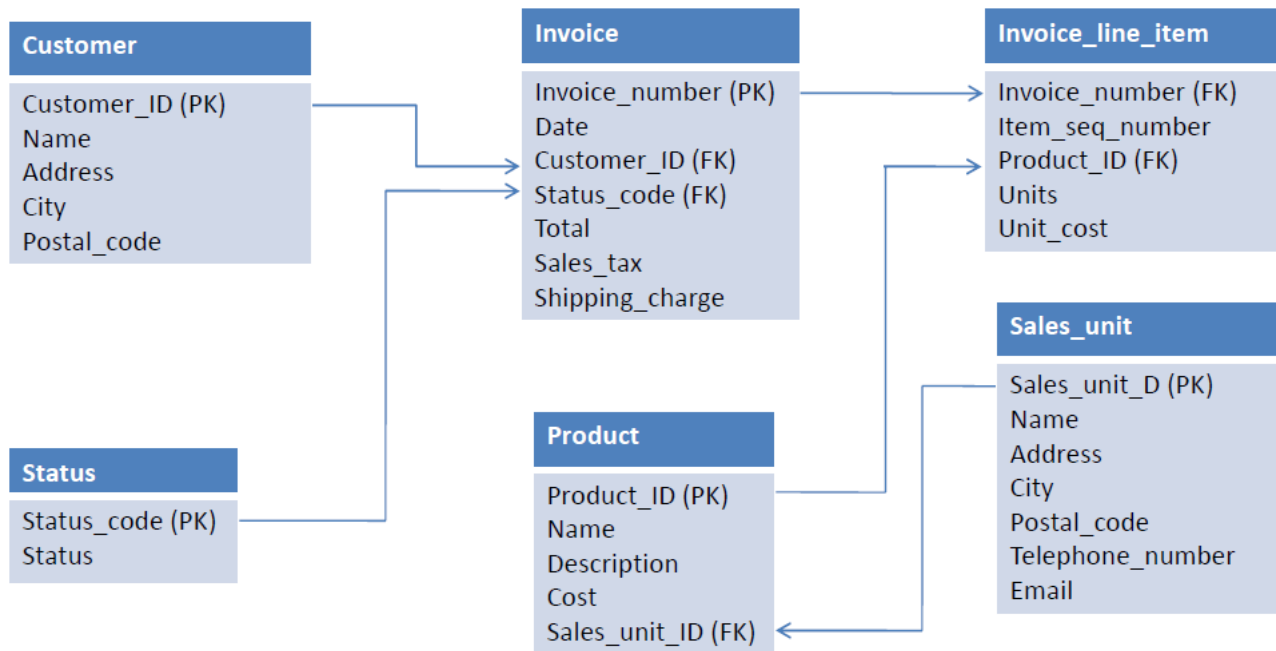
Are ideal for everyday operation

Fail at analytical tasks

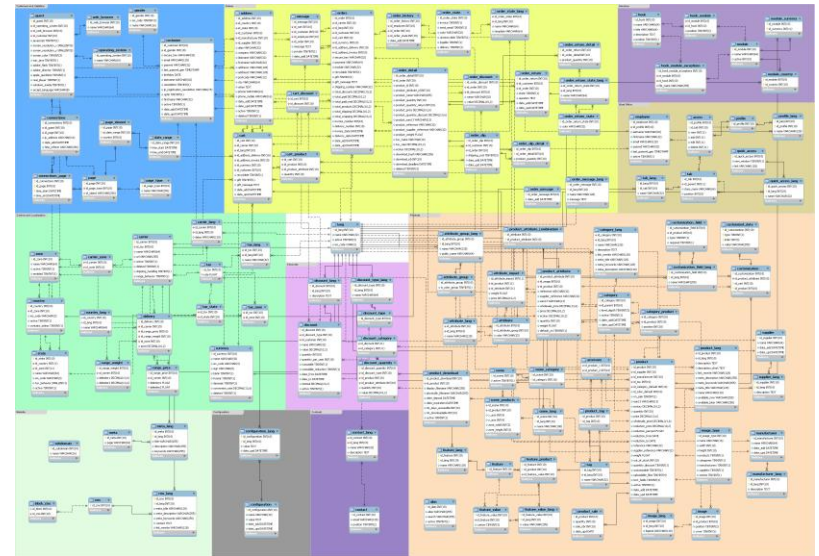
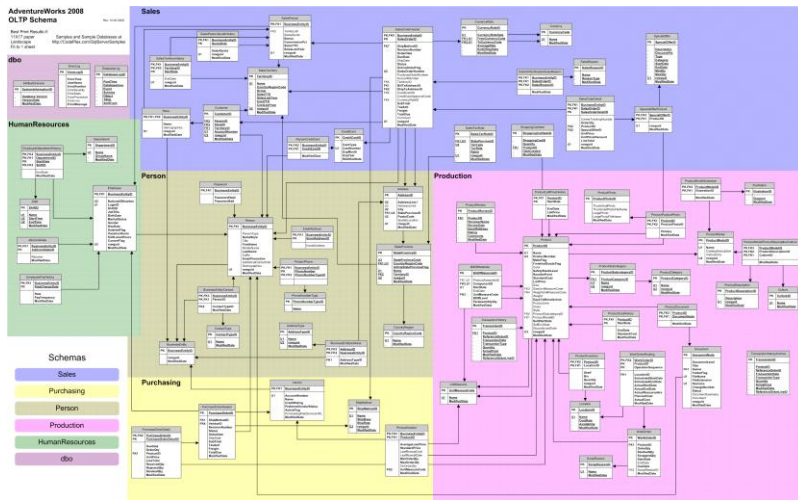
What is the problem here ?

OLTP

Typical operational DB:



OLTP



OLAP

Analytical Systems (OLAP)

on-line analytical processing

Analytical Database Systems

Goal:

supporting decision makers

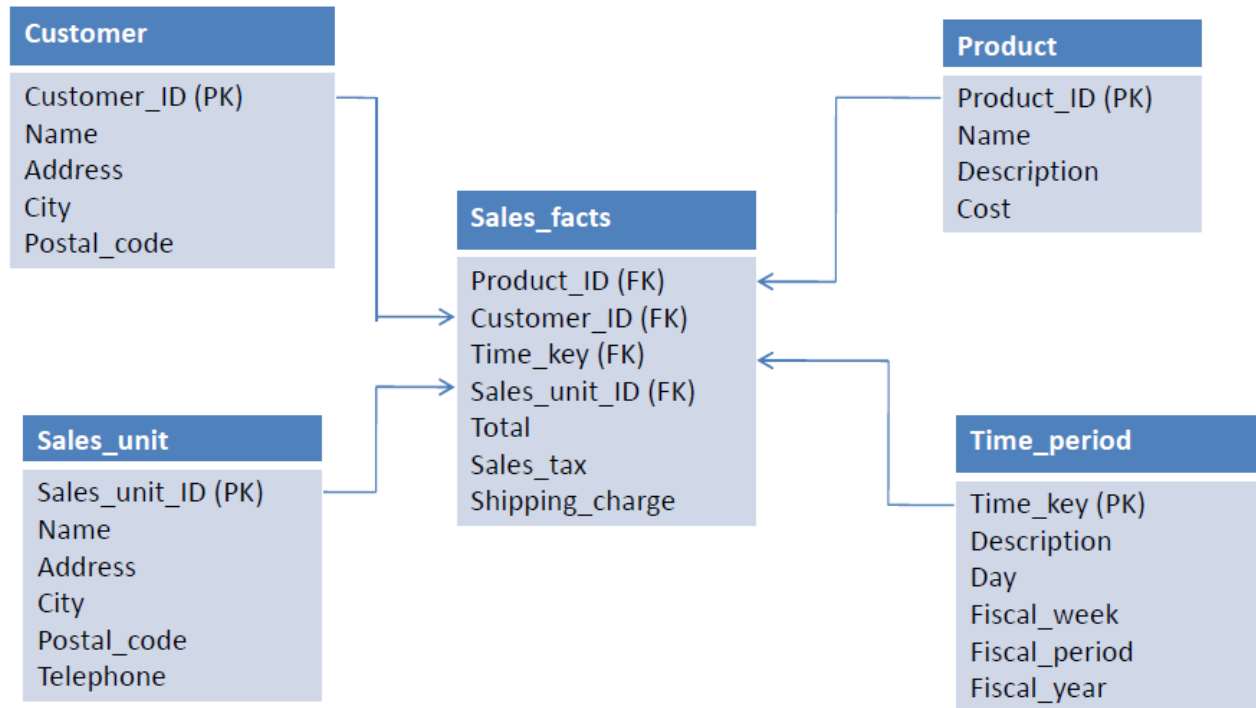
Strategic usage of data

Main tasks:

*effective processing of multidimensional
queries concerning huge volumes of data*

OLAP

Typical OLAP DW:



OLAP

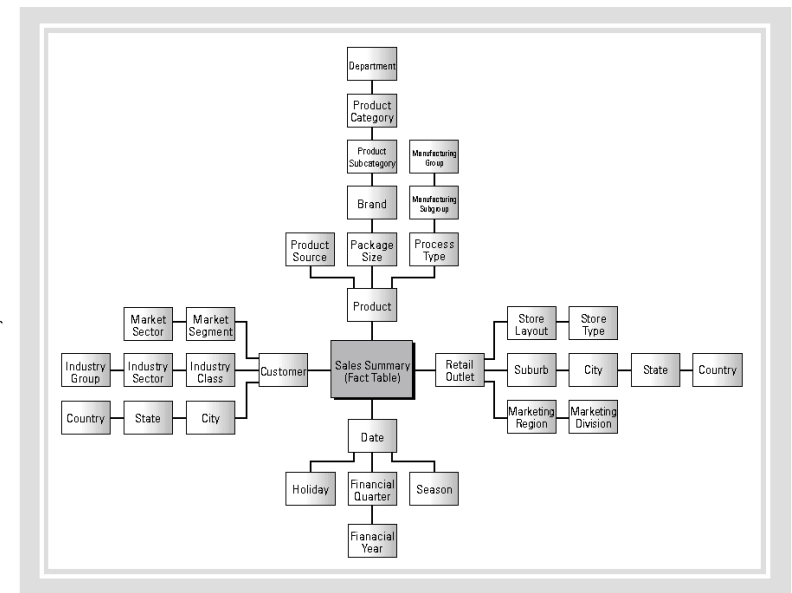
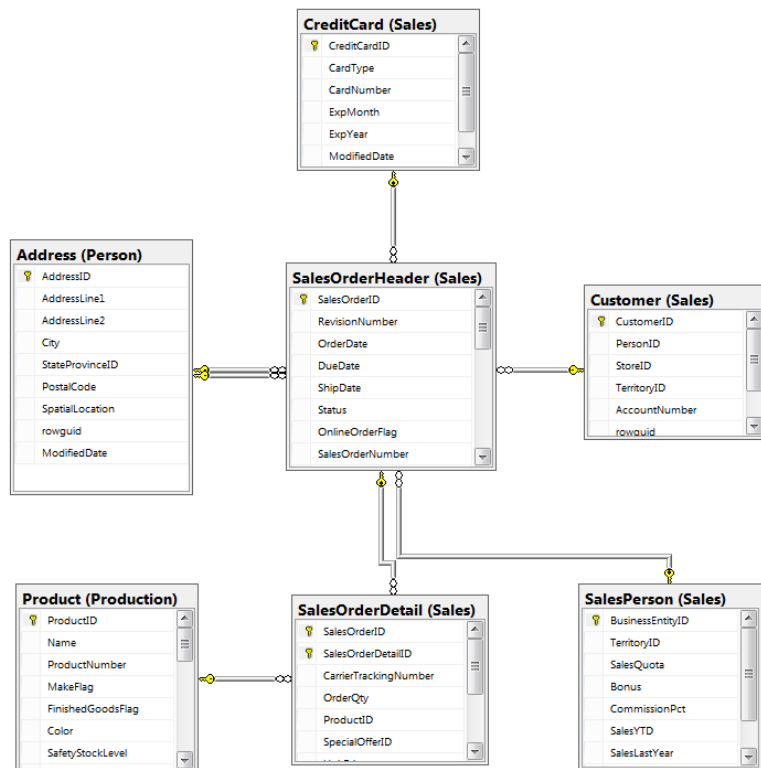


Figure 4. Snowflake Schema

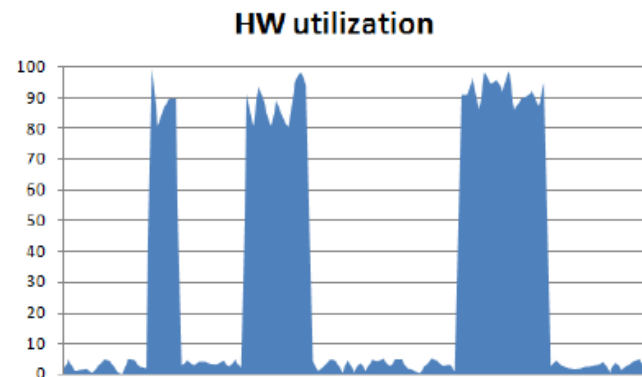
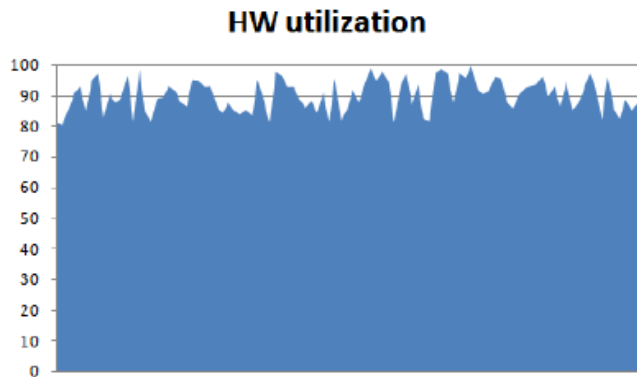
OLTP/OLAP

Bus vs. Train Analogy

Regular intense schedule, short paths, multiple entities

Less intense schedule, long paths, sporadic entities

Utilisation:



OLTP/OLAP

	OLTP	OLAP
User	Anyone	Knowledge worker
Function	Day-to-day operation	Decision support
DB design	Application-oriented	Subject-oriented
Data	Current, up-to-date, detailed, highly normalised	Historical, summarised, integrated, consolidated
Access	Read/write, multiple simul. users	Lots of scans, usually single user
Unit of work	Simple transaction	Complex query
DB size	MB-GB	GB-TB-PB (ZB)
Metric	Transaction throughput	Query throughput, response
update	continuous	Sporadic, requires pre-procesing
lifetime	short	long

OLTP/OLAP

	OLTP	OLAP
Tuning	Highly tuned for frequent access to small amounts of data	Tuned for infrequent access to larger quantities of data
Data retention	Data retained to meet daily requirements (90 days, 1 year)	Data retained longer to support historical reporting, comparison, etc.
Queries	Pre-cooked (predefined application queries)	Ad-hoc, complex and interactive reporting queries (access to multiple records, multiple joins, aggregations)
Retrieval	Detail row retrieval	Aggregation and group by
Design requirements	Known	Vague
Critical to	Daily business operation	Management decision that may affect profitability

OLTP/OLAP



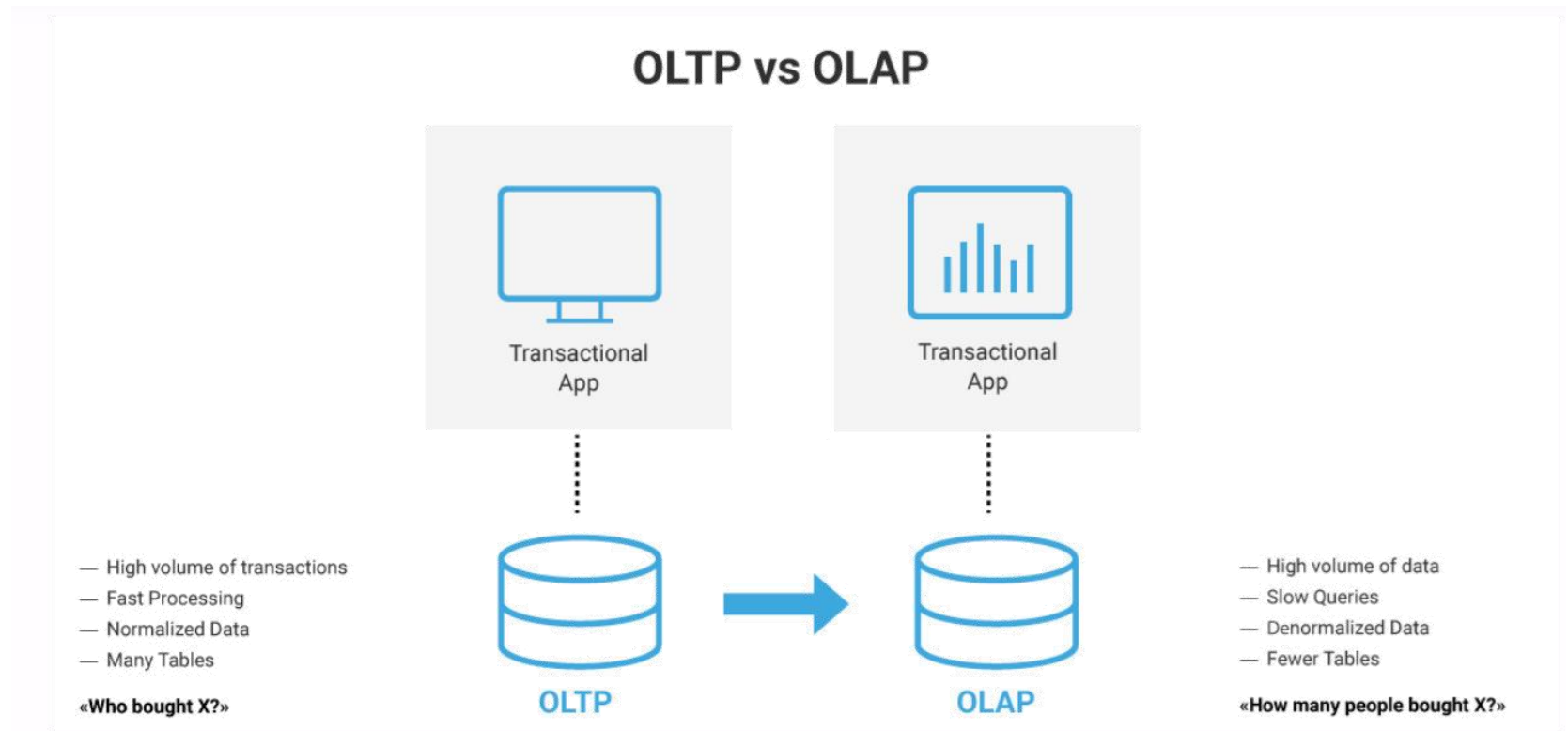
OLTP characteristics:

- Many transactions
- Latency sensitive
- Small payloads
- Balanced read/write or Heavy write workloads

OLAP characteristics:

- Few transactions
- Throughput sensitive
- Large (return) payloads
- Heavy read workloads (including full table scans)

OLTP/OLAP



OLAP: Understanding the Past and Present

Purpose:

OLAP is primarily focused on analyzing historical and current data to understand trends, patterns, and relationships. It's about gaining insights from existing data to inform business decisions.

Methodology:

OLAP uses multidimensional data models (like data cubes) to organize and summarize data, allowing users to quickly query and analyze data from different perspectives (dimensions).

Common OLAP operations include: Slicing and Dicing, Drill-down and Roll-up, Aggregation

Output:

OLAP typically produces reports, dashboards, and visualizations that help business users understand past performance, current status, and identify areas for improvement.

Focus:

Descriptive analytics - "What happened?" and "Why did it happen?"

ML: Predicting the Future and Automating Decisions

Purpose:

Machine Learning is focused on building models that can learn from data to make predictions, classify data, or automate tasks. It's about using data to anticipate future outcomes and make intelligent decisions.

Methodology:

ML algorithms learn patterns from training data and then use these patterns to make predictions or classifications on new, unseen data.

Common ML tasks include: Classification, Regression, Clustering, Anomaly Detection

Output:

ML models produce predictions, classifications, recommendations, and automated actions

Focus:

Predictive and Prescriptive analytics - "What will happen?" and "What should we do?"



Operational database is CRUCIAL

Operational database is NOT ENOUGH

An abstract graphic on the left side of the slide. It features three rows of 3D rectangular blocks in red, green, and blue. From each block, several horizontal lines of the same color extend to the right, creating a sense of motion or data flow. The blocks and lines are arranged in a staggered, overlapping manner.

Necessity Is the Mother of Invention

At first, the same databases were used for both transactional processing and analytic queries

SQL is quite flexible in this regard

Late 80's/90's move to dedicated solutions focused on data analysis

Information wise - **Data warehousing**

Data warehousing - DW

Nowadays

DW now exists in almost all larger enterprises

- multiple OLTP systems – often geographically distributed
- large amounts of data

In smaller companies DW are less popular

- do not have many OLTP systems
- smaller amounts of data
- some tools are really expensive

Vendors to know:

Microsoft, Amazon, Cloudera, IBM, SAP, Snowflake, Teradata, Snowflake, Oracle, etc.

Recent trends

SQL on Hadoop

Apache Hive, Apache Spark, Apache Kudu, Cloudera Impala, Facebook Presto, Apache Drill, Apache Kylin, etc.

Cloud solutions

AWS Redshift, Google BigQuery, Azure Synapse Analytics, Snowflake, etc.

Data Lake and Data Lakehouses

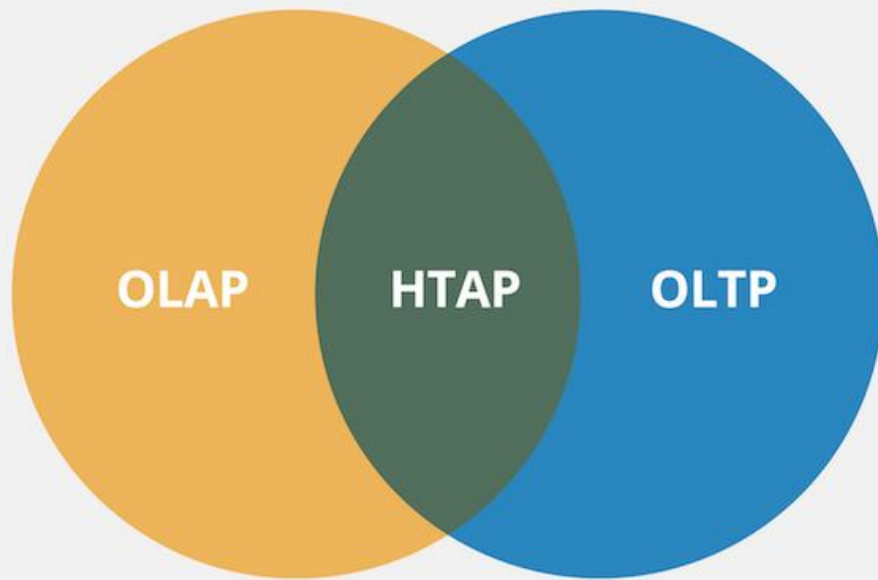
Dremio, Kylo, CDP, etc.

Virtualization

Denodo, Qubera, etc.

Real-time and streaming data:

ClickHouse, Apache Druid, Materialise



HTAP

Handling so-called operational analytical processing workloads

In a way real-time analytics

Online analytical processing

A profile for software applications that enables users to analyze data multidimensionally, via rolling up, slicing, dicing and drilling down. OLAP databases are for historical analytics, frequently on transactional data.

Hybrid transaction/analytical processing

A technology for managing live operational data and historical data within the same database management system, instead of using separate OLTP databases that feed into an OLAP data warehouse. HTAP enables both real-time analytics for processing transactions and historical analytics.

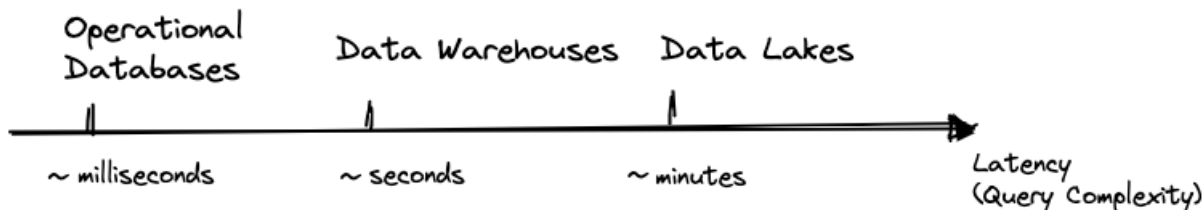
Online transaction processing

OLTP databases are built to process mission-critical transactions for the business's operations (e.g., logging orders, pulling customer account information etc.). Generally, OLTP databases feed information to and receive data from mobile applications, point of sale systems, web applications, websites, ATM machines etc.

Do not confuse with operational reporting

act of detailing a company's day-to-day operations using visuals that show operations workflow to make fast, informed decisions about daily operations.

Example – TiDB, AlloyDB



source:
www.softwareadvice.com/resources/olap-data-warehouse-alternatives/



Queries OLAP perspective



Querying OLAP

OLAP query languages

Getting from **OLAP** to the data

As in the relational model, through queries

In OLTP we have SQL as the standard query language

However, OLAP operations are hard to express in SQL

There is no standard query language for OLAP

Though MDX is somewhat popular

Typical reporting/analytical OLAP applications requires **cross-tab, pivots**

Multidimensional data

Transactional data

	Order...	OrderDate	OrderA...	CustomerNa...
1	1	2012-03-01	10.0000	Joe
2	2	2012-03-01	11.0000	Sam
3	3	2012-03-02	10.0000	Beth
4	4	2012-03-02	15.0000	Joe
5	5	2012-03-02	17.0000	Sam
6	6	2012-03-03	12.0000	Joe
7	7	2012-03-04	10.0000	Beth
8	8	2012-03-04	18.0000	Sam
9	9	2012-03-04	12.0000	Joe
10	10	2012-03-04	11.0000	Beth
11	11	2012-03-05	14.0000	Sam
12	12	2012-03-06	17.0000	Beth
13	13	2012-03-06	19.0000	Joe
14	14	2012-03-07	13.0000	Beth
15	15	2012-03-07	16.0000	Sam

Identified by transaction keys
each data item is identified
by a unique key

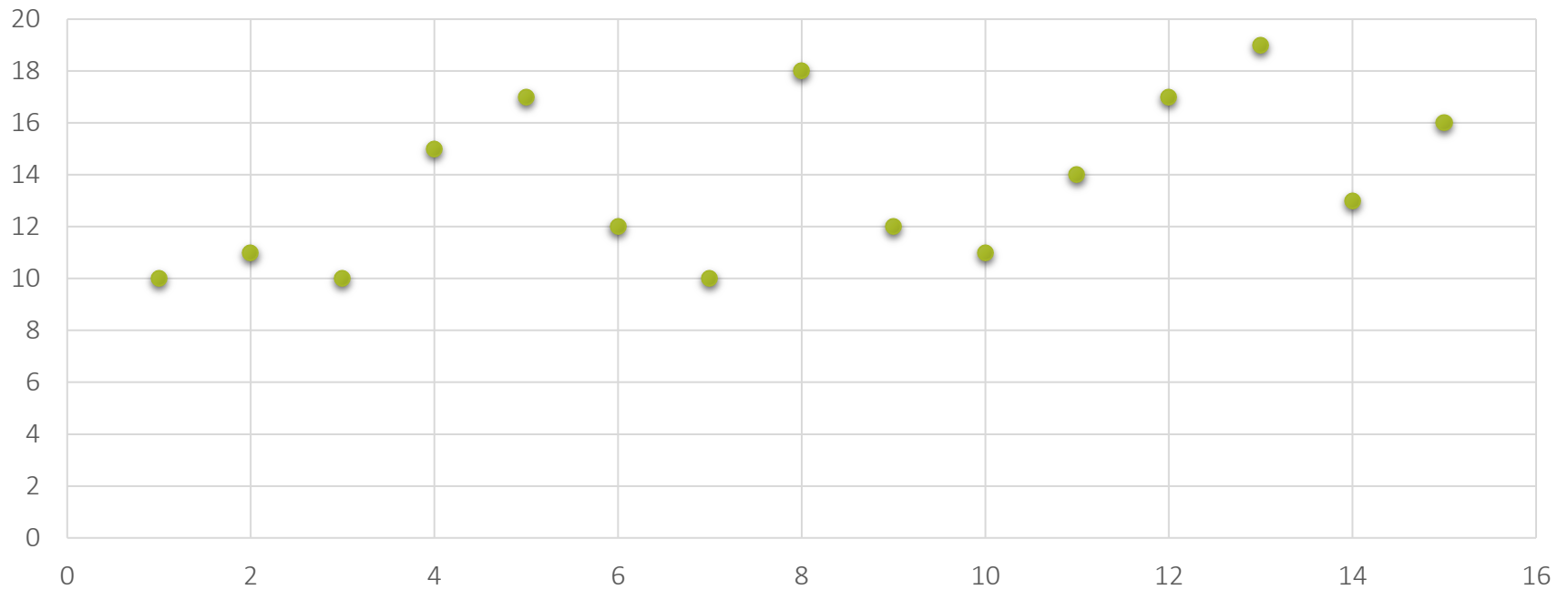
unique key → *attribute*,
attribute, *attribute*

1-dimensional structure

A point in such a space
Is defined by a unique key

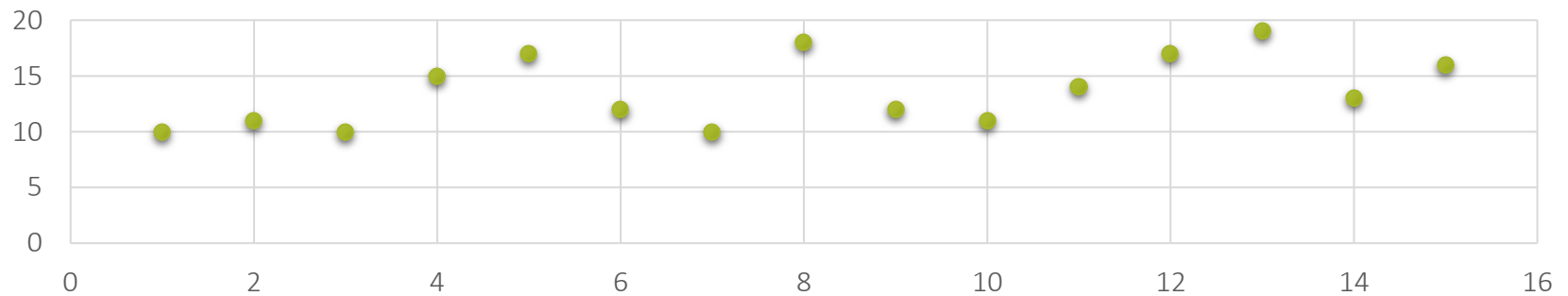
Transactional Data

Order Amount / ID

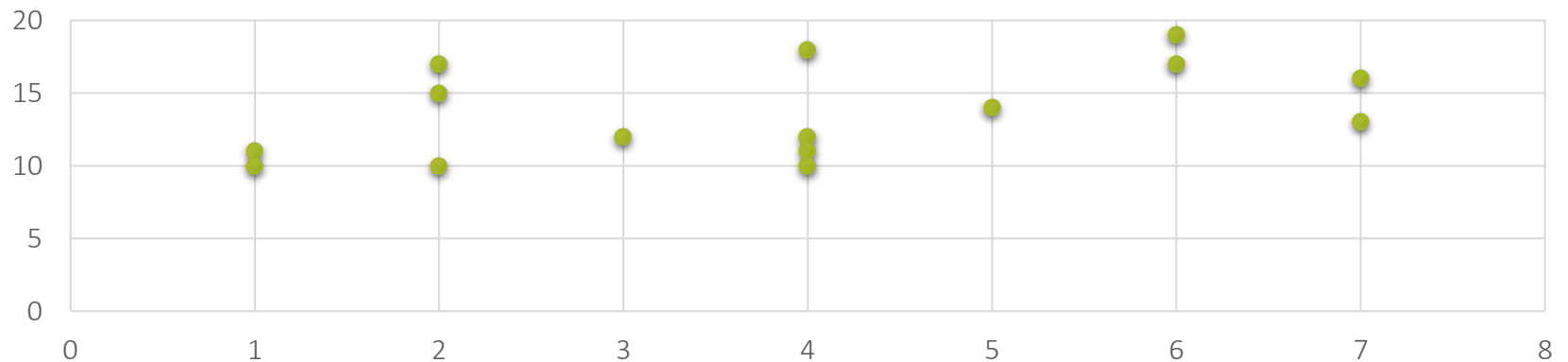


Transactional Data

Order Amount / ID

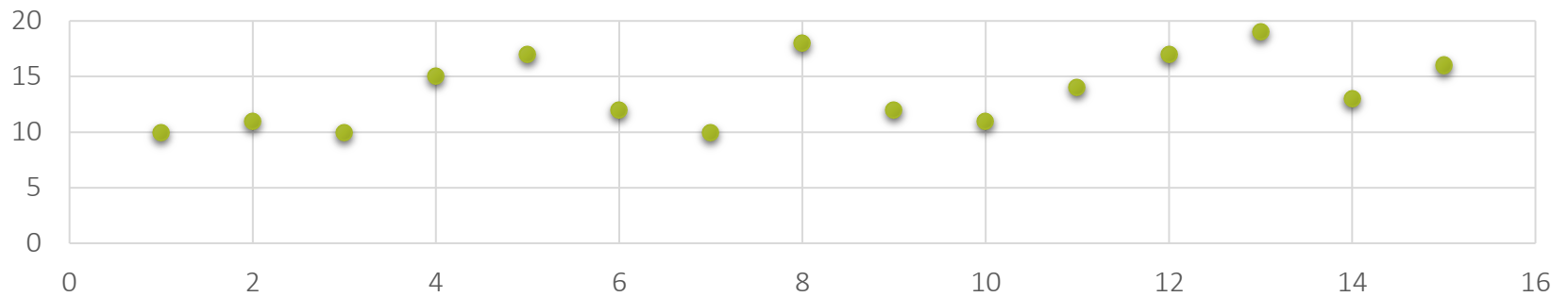


Order Amount / Day

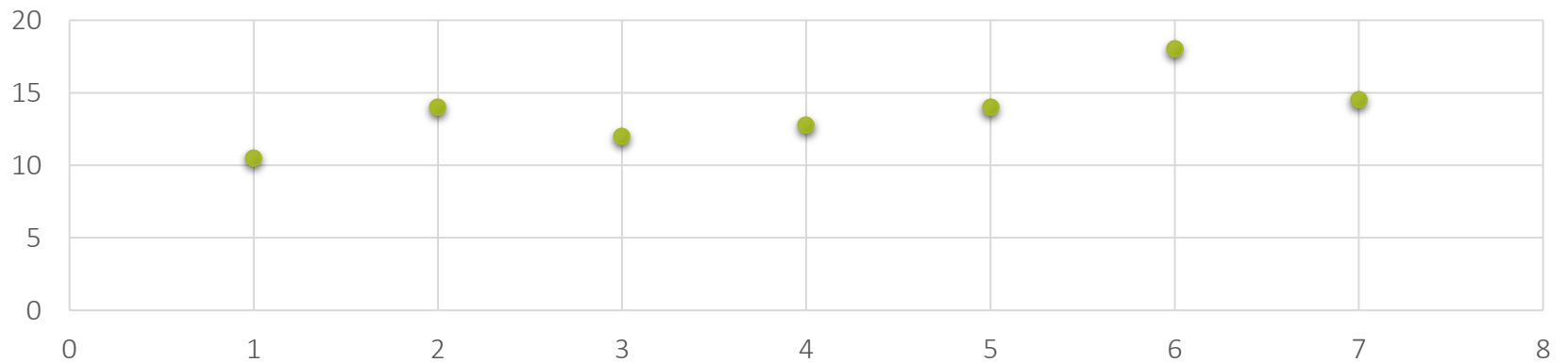


Transactional Data

Order Amount / ID



AVG(Order Amount) / Day



Analytical Data

Analytical Data

Each data item is identifiable by a unique set of dimension values

$dim1, dim2, \dots, dimN \rightarrow measure$

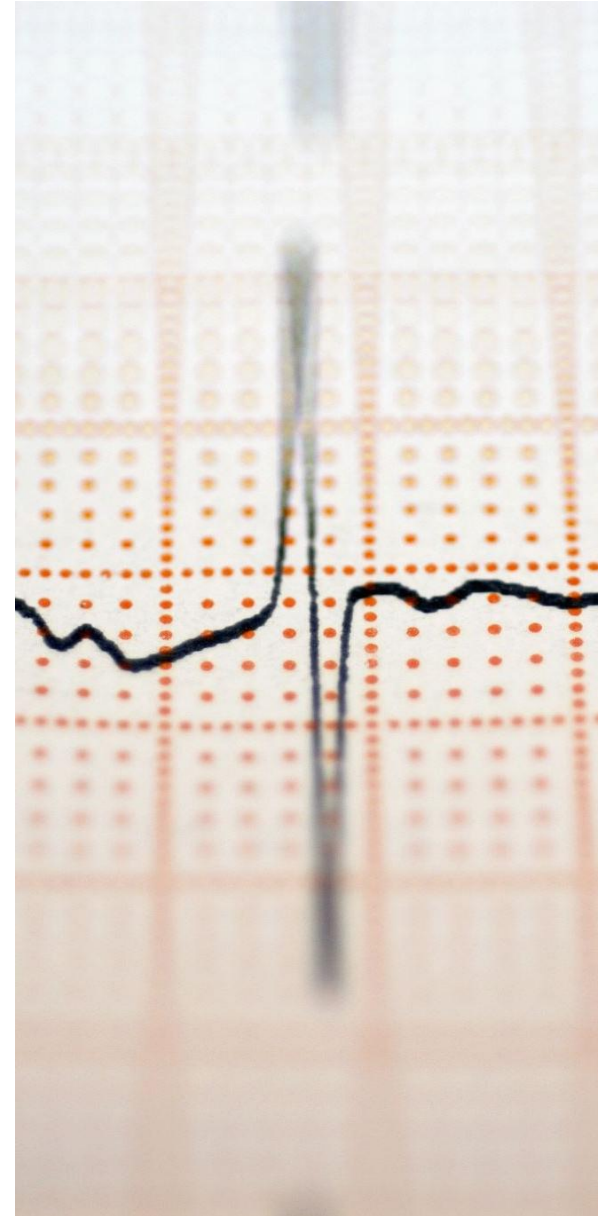
$dim1, dim2, \dots, dimN \rightarrow measure1, \dots, measureM$

N-dimensional structure

Point in such a structure

Is identified by an intersection of dimensions

Dimensions' values



Analytical Data

P1	100	105	...	
P2	0	120	...	
...	...	...	...	
	C1	C2	...	

Analytical Data

Pivot Table

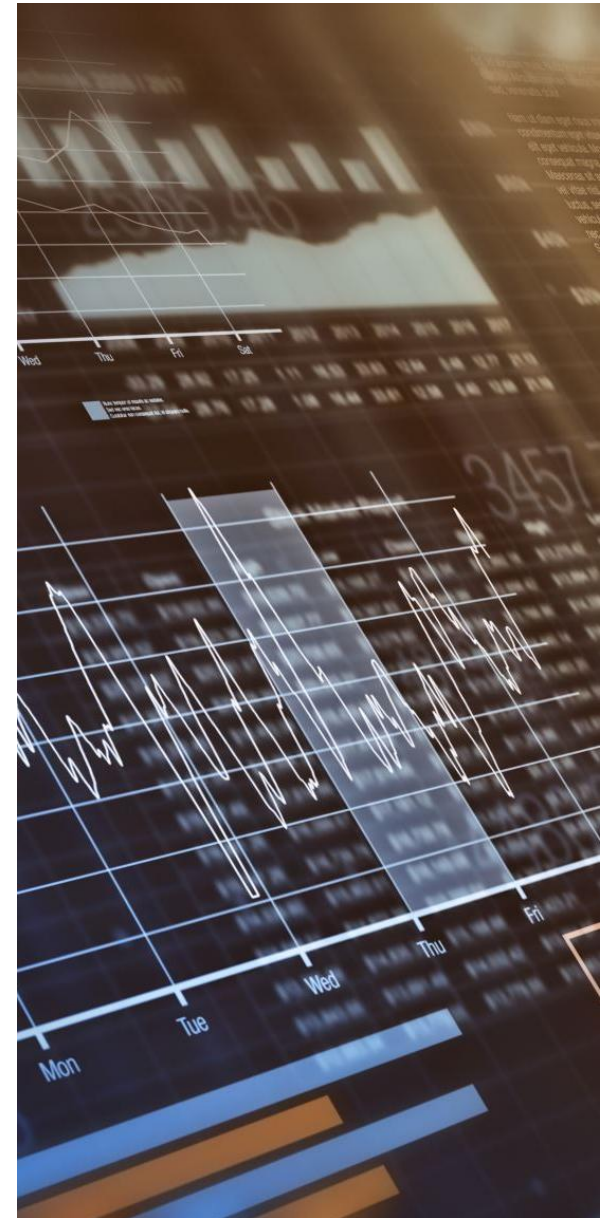
Pivot tables allow you to quickly summarize and analyze complex data

Follows the analytical stance on data
dimension-wise rather than transaction-wise

Dynamic creation of aggregations and summaries

What if the source data is in transactional form ?

Enforce OLAP perspective



Analytical Data

T1	T2	*	0	120	...	*
	P1		0	0	...	0
	P2		0	120		120
	*	100	105	...	*	..
	P1	100	105	...	205	
	P2	0	0	...	0	*
	...	...	...	...	...	
		C1	C2	...	*	

Analytical Data

Analytical data:

Product	Customer	Time	Sales
P1	C1	T1	100
P1	C2	T1	105
P2	C2	T2	120

T2	*	0	120	...	*
	P1	0	0	...	0
	P2	0	120	...	120
	...	...	...	...	...
T1	*	100	105	...	*
	P1	100	105	...	205
	P2	0	0	...	0
	...	...	...	...	...
		C1	C2	...	*



Analytical Data

Analytical data:

Product	Customer	Time	Sales
P1	C1	T1	100
P1	C2	T1	105
P2	C2	T2	120

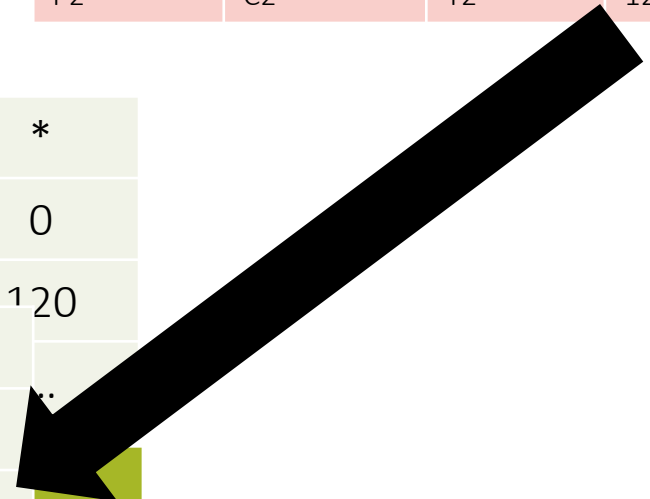
T2	...	0	120	...	*
	P1	0	0	...	0
	P2	0	120	...	120
T1	*	100	105	...	*
	P1	100	105	...	205
	P2	0	0	...	0
	...	...	...	...	...
	C1	C2	...	*	

Analytical Data

Analytical data:

Product	Customer	Time	Sales
P1	C1	T1	100
P1	C2	T1	105
P2	C2	T2	120

T2	*	0	120	...	*
	P1	0	0	...	0
	P2	0	120	...	120
	...	...	...	...	...
T1	*	100	105	...	*
	P1	100	105	...	205
	P2	0	0	...	0
	...	...	...	...	...
		C1	C2	...	*



Analytical Data

Analytical data:

Product	Name	...
P1	P1	...
P1	P1	...
P2	P2	...

Customer	FirstName	...
C1	C1	...
C2	C2	...
C2	C2	...

Product	Customer	Time	Sales
P1	C1	T1	100
P1	C2	T1	105
P2	C2	T2	120



T2	*	0	120	...	*
	P1	0	0	...	0
	P2	0	120	...	120
	...	...	...	...	...
T1	*	100	105	...	*
	P1	100	105	...	205
	P2	0	0	...	0
	...	...	...	...	...
		C1	C2	...	*

Analytical Data

Pivot Table in Excel

Rows

Columns

Values

Pages / Filters

Drag fields between areas below:

FILTERS	COLUMNS
ROWS	VALUES

☐ Defer Layout Update UPDATE

Pivot Table				
Staff Name	James Smith			
Sum of Hours				
	Non-Teaching	Private Lessons	Grand Total	
2019-01-19	0.25	5.5	5.75	
2019-01-26	0.17	5.5	5.67	
2019-01-22	0.25	4.25	4.5	
2019-01-29	0.17	4.25	4.42	
2019-01-17	0.25	3.75	4	
2019-01-24	0.25	3.75	4	
2019-01-31	0.17	3.75	3.92	
Grand Total	1.51	30.75	32.26	

PivotTable Fields

FIELD NAME

☒ Staff Name
☐ Service

Filters
: Staff Name ?
: Service Catego... ?

Columns
: Service Catego... ?

Rows
: Service Date ?
: Sum of Hours ?

Values
: Sum of Hours ?

Drag fields between areas

Analytical Data

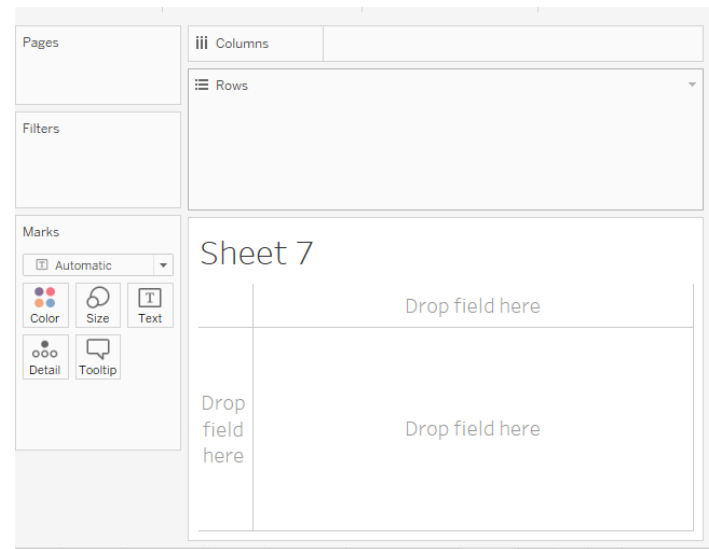
Pivot Table in Tableau

Rows

Columns

Values

Pages / Filters



The screenshot shows the Tableau interface with a pivot table named 'Sheet 8'. The 'Columns' shelf contains 'Color' and the 'Rows' shelf contains 'Name (ProductCategory)1'. The 'Marks' shelf is set to 'Automatic' and contains 'AVG(Profit)'. The pivot table displays the following data:

	Color									
Name (Prod..	Null	Black	Blue	Grey	Multi	Red	Silver	Silve..	White	Yell..
Null	1.9	0.0					0.0			
Accessories	18.3	21.9	21.9	73.4		21.9	34.4			
Bikes		610.2	527.1			767.1	737.3			529.1
Clothing		32.3	39.8		25.8				5.9	12.4
Components	43.6	237.0	252.9			311.8	245.9	35.6		246.1

Analytical Data

Pivot Table in Power BI

Matrix visualization

Rows

Columns

Values

Filters

Region	Central		East		West		Total	
Sales Stage	Opportunity Count	Revenue	Opportunity Count	Revenue	Opportunity Count	Revenue	Opportunity Count	Revenue
Proposal								
Small	8	\$13,095,186	3	\$4,770,862	3	\$3,720,287	14	\$21,586,335
Medium	4	\$15,283,161	6	\$25,607,581	5	\$21,456,937	15	\$62,347,679
Large	2	\$18,344,522	4	\$29,592,481	2	\$17,855,445	8	\$65,792,448
Total	14	\$46,722,869	13	\$59,970,924	10	\$43,032,669	37	\$149,726,462
Total	14	\$46,722,869	13	\$59,970,924	10	\$43,032,669	37	\$149,726,462

The screenshot shows the Power BI interface. The **Visualizations** pane on the left has the **Pivot Table** icon selected. The **Fields** pane on the right shows a list of data fields. The **Rows** section of the Pivot Table is highlighted with a red box and contains **Sales Stage** and **Opportunity Size**. The **Columns** section contains **Region**. The **Values** section contains **Opportunity Count** and **Revenue**. The **Fields** pane shows a search bar and a list of fields including **Account**, **Fact**, **Account ID**, **Avg Opportunity...**, **Avg Revenue**, **Create Date**, **Date**, **EstimatedCloseD...**, **Factored Revenue**, **FactoredProduct...**, **Month**, **Month_Number**, **Opportunity Cou...**, **Opportunity Days**, **Opportunity ID**, **Partner ID**, **Product ID**, **ProductRevenue**, **Revenue**, **Sales Stage ID**, **Tot Opportunity ...**, and **Year**. The **Opportunity** field is selected in the **Fields** pane.



Querying OLAP

Using SQL for OLAP

Querying OLAP

Shortcomings of SQL/92 with regard to OLAP queries

Hard or impossible to express in SQL

Multiple aggregations

Comparisons (with aggregation)

Reporting features

Performance penalty

*Poor execution of queries with many AND
and OR conditions*

Return data

Relation

SQL3

SQL-99 → SQL 2003 (SQL3)

Prepare SQL for OLAP queries

New SQL commands

GROUPING SETS

ROLLUP

CUBE

New aggregate functions

Queries of type “top k”

SQL and T-SQL

What is Standard SQL?

- Standard SQL, usually referred to simply as “SQL,” is a type of programming language called a query language used for communicating with a database.
- SQL is used for adding, retrieving, or updating data stored in a database. It is used across many different types of databases.
- Despite the clear and rigid specifications of standard SQL, it does allow for database companies to add their own extensions to set them apart from other products.

What is T-SQL?

- T-SQL, which stands for Transact-SQL and is sometimes referred to as TSQL, is an extension of the SQL language used primarily within Microsoft SQL Server.
- This means that it provides all the functionality of SQL but with some added extras.
- You can think of it a bit like a SQL dialect — it’s very similar to regular SQL, but it has a few extras and differences that make it unique.

T-SQL

General differences:

- proprietary, extension, procedural

Besides

SQL and T-SQL also have some slightly different command key words

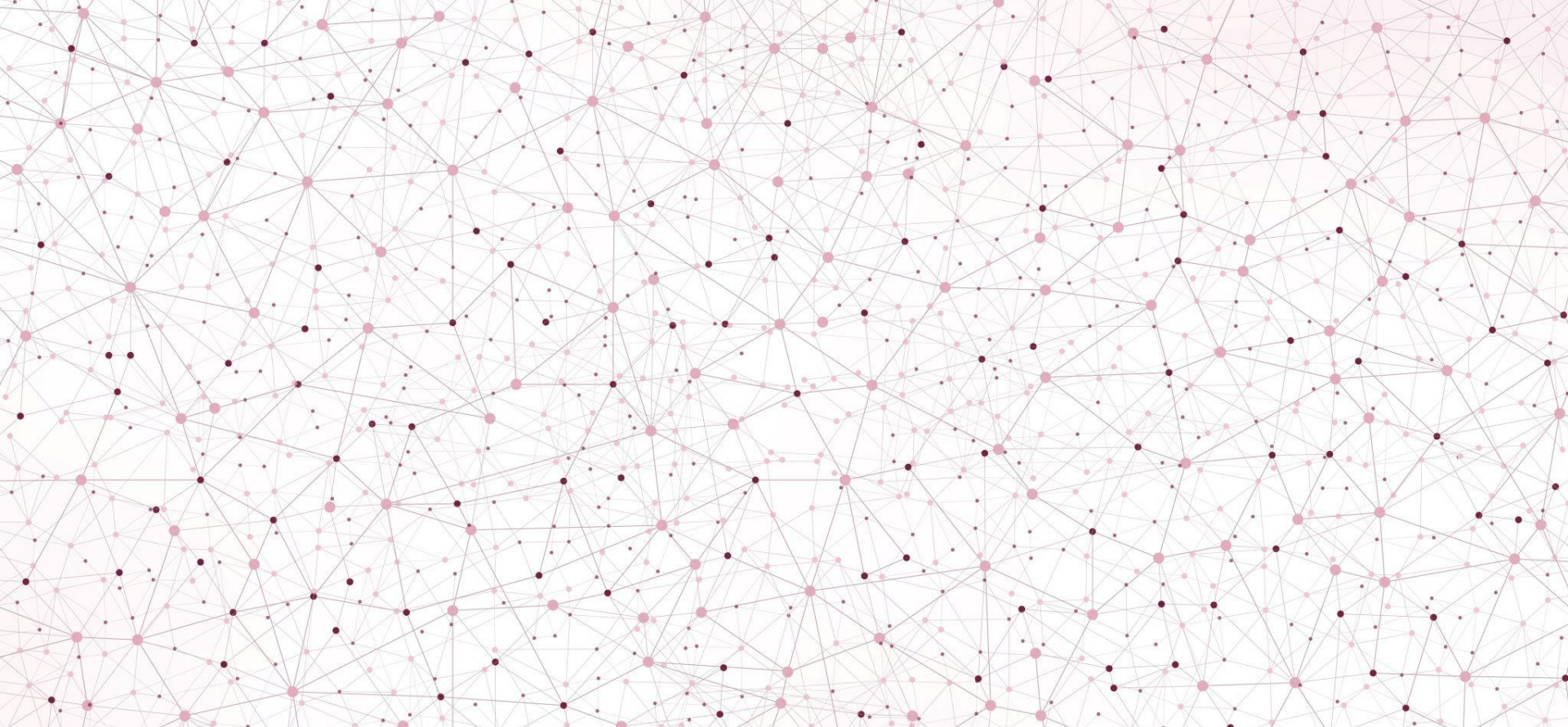
Like TOP instead of LIMIT

T-SQL also features functions that are not part of regular SQL

Like ISNULL()

Take a quick look at 14 basic differences

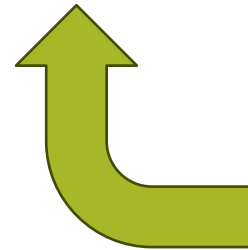
<https://learnsql.com/blog/14-differences-sql-vs-tsql/>



Pivot and Case

Pivot

	A	B	C	D	E	F
1	Respondent	Question 1	Question 2	Question 3	Question 4	Question 5
2	Ken Flerlage	1	4	4	5	2
3	John Smith	5	4	3	3	5
4	Willy Wonka	3	2	5	1	2
5	Fyodor Dostoevsky	5	2	2	1	
6	George Orwell	1	5	5	3	



	A	B	C
1	Respondent	Question	Answer
2	Ken Flerlage	Question 1	1
3	Ken Flerlage	Question 2	4
4	Ken Flerlage	Question 3	4
5	Ken Flerlage	Question 4	5
6	Ken Flerlage	Question 5	2
7	John Smith	Question 1	5
8	John Smith	Question 2	4
9	John Smith	Question 3	3
10	John Smith	Question 4	3
11	John Smith	Question 5	5
12	Willy Wonka	Question 1	3
13	Willy Wonka	Question 2	2
14	Willy Wonka	Question 3	5
15	Willy Wonka	Question 4	1
16	Willy Wonka	Question 5	2

Pivot tables

	A	B	C	D	E
1	Pivot Table				
2	Staff Name	James Smith			
3					
4	Sum of Hours				
5		Non-Teaching	Private Lessons	Grand Total	
6	2019-01-19	0.25	5.5	5.75	
7	2019-01-26	0.17	5.5	5.67	
8	2019-01-22	0.25	4.25	4.5	
9	2019-01-29	0.17	4.25	4.42	
10	2019-01-17	0.25	3.75	4	
11	2019-01-24	0.25	3.75	4	
12	2019-01-31	0.17	3.75	3.92	
13	Grand Total	1.51	30.75	32.26	
14					
15					
16					

Desired Result
Pivot Table
Source Table
+

PivotTable Fields

FIELD NAME

Search fields

☒ Staff Name

☐ Service

Filters

Staff Name

Columns

Service Catego...

Rows

Service Date

Values

Sum of Hours

Drag fields between areas

Drag fields between areas below:

FILTERS

COLUMNS

ROWS

VALUES

☐ Defer Layout Update

UPDATE

Pivot

SELECT ...

FROM data_source

PIVOT (

aggregate

FOR col_to_pivot

IN values_pivoted

)

WHERE...

	A	B	C	D	E	F
1	Respondent	Question 1	Question 2	Question 3	Question 4	Question 5
2	Ken Flerlage	1	4	4	5	2
3	John Smith	5	4	3	3	5
4	Willy Wonka	3	2	5	1	2
5	Fyodor Dostoevsky	5	2	2	1	
6	George Orwell	1	5	5	3	



	A	B	C
1	Respondent	Question	Answer
2	Ken Flerlage	Question 1	1
3	Ken Flerlage	Question 2	4
4	Ken Flerlage	Question 3	4
5	Ken Flerlage	Question 4	5
6	Ken Flerlage	Question 5	2
7	John Smith	Question 1	5
8	John Smith	Question 2	4
9	John Smith	Question 3	3
10	John Smith	Question 4	3
11	John Smith	Question 5	5
12	Willy Wonka	Question 1	3
13	Willy Wonka	Question 2	2
14	Willy Wonka	Question 3	5
15	Willy Wonka	Question 4	1
16	Willy Wonka	Question 5	2

Pivot

PIVOT follows the following process to obtain the output result set:

Performs a GROUP BY on its *input_table*

against the grouping columns and produces one output row for each group

grouping columns in the output row obtain the corresponding column values for that group in the input_table.

Generates values for the columns in the column list

for each output row by performing the following:

Grouping additionally the rows generated in the GROUP BY in the previous step against the pivot_column.

For each output column in the column_list, selecting a subgroup that satisfies the condition:

`pivot_column = CONVERT(<data type of pivot_column>, 'output_column')`

Aggregate_function is evaluated against the value_column on this subgroup and its result is returned as the value of the corresponding output_column.

If the subgroup is empty, SQL Server generates a null value for that output_column.

If the aggregate function is COUNT and the subgroup is empty, zero (0) is returned.

Pivot

JOB_ID	Y	SALARY
1 AD_PRES	2013	24000
2 AD_VP	2015	17000
3 AD_VP	2011	17000
4 IT_PROG	2016	9000
5 IT_PROG	2017	6000
6 IT_PROG	2015	4800
7 IT_PROG	2016	4800
8 IT_PROG	2017	4200
9 FI_MGR	2012	12008
10 FI_ACCOUNT	2012	9000
11 FI_ACCOUNT	2015	8200
12 FI_ACCOUNT	2015	7700
13 FI_ACCOUNT	2016	7800
14 FI_ACCOUNT	2017	6900
15 PU_MAN	2012	11000
16 PU_CLERK	2013	3100
17 PU_CLERK	2015	2900
18 PU_CLERK	2015	2800
19 PU_CLERK	2016	2600
20 PU_CLERK	2017	2500
21 ST_MAN	2014	8000
22 ST_MAN	2015	8200

JOB_ID	Y	SUM(SALARY)
1 AD_PRES	2013	24000
2 AD_VP	2015	17000
3 AD_VP	2011	17000
4 IT_PROG	2016	13800
5 IT_PROG	2017	10200
6 IT_PROG	2015	4800
7 FI_MGR	2012	12008
8 FI_ACCOUNT	2012	9000
9 FI_ACCOUNT	2015	15900
10 FI_ACCOUNT	2016	7800
11 FI_ACCOUNT	2017	6900
12 PU_MAN	2012	11000
13 PU_CLERK	2013	3100
14 PU_CLERK	2015	5700
15 PU_CLERK	2016	2600
16 PU_CLERK	2017	2500
17 ST_MAN	2014	8000
18 ST_MAN	2015	14700
19 ST_MAN	2013	7900

Założmy, iż chcemy wyznaczyć sumę wynagrodzeń dla poszczególnych departamentów oraz etatów:

Spróbujemy przedstawić dane poprzez przeniesienie etatów do kolumn

Dane do PIVOT:

```
SELECT job_id, EXTRACT(YEAR from  
hire_date) Y, salary  
FROM employees;
```

Po grupowaniu:

```
SELECT job_id, EXTRACT(YEAR from hire_date)  
Y, SUM(salary)  
FROM employees  
GROUP BY EXTRACT(YEAR from hire_date),  
job_id;
```

Results		Messages	
	CustomerID	TotalDue	QR
1	29825	23153,2339	2
2	29672	1457,3288	2
3	29734	36865,8012	2
4	29994	32474,9324	2
5	29565	472,3108	2
6	29898	27510,4109	2
7	29580	16158,6961	2
8	30052	5694,8564	2
9	29974	6876,3649	2
10	29614	40487,7233	2
11	29747	807,2585	2
12	29566	6893,2549	2
13	29890	9153,6054	2
14	30067	6895,41	2
15	29844	4216,0258	2
16	29596	2955,0542	2
17	29827	6434,0848	2

Pivot

Assume we want to analyse *sales by customer by different quarters*:

It is easier if we somehow move quarters to columns

Data for Pivot:

```
SELECT
    CustomerID, TotalDue, DATEPART(quarter, OrderDate) AS QR
FROM Sales.SalesOrderHeader
```

After grouping:

```
SELECT
    CustomerID, Sum(TotalDue), DATEPART(quarter, OrderDate) AS QR
FROM Sales.SalesOrderHeader
GROUP BY
    CustomerID, DATEPART(quarter, OrderDate)
```

Pivot

```
SELECT CustomerID, Sum(TotalDue),
DATEPART(quarter, OrderDate) AS QR
FROM Sales.SalesOrderHeader
GROUP BY CustomerID, DATEPART(quarter,
OrderDate)
```

```
SELECT CustomerID, [1], [2], [3], [4]
```

```
FROM
```

```
(SELECT CustomerID, TotalDue, DATEPART(quarter, OrderDate) AS QR
FROM Sales.SalesOrderHeader
) AS SourceTable
```

```
PIVOT (
```

```
Sum (TotalDue)
FOR QR IN ( [1],[2],[3],[4] )
) AS PivotTable
```

```
Order by CustomerID
```

	CustomerID	TotalDue	QR
1	29825	23153.2339	2
2	29672	1457.3288	2
3	29734	36865.8103	2
4	29994	32474.9324	2
5	29565	472.3108	2
6	29898	27510.4109	2
7	29580	16158.6961	2
8	30052	5694.8564	2
9	29974	6876.3649	2
10	29614	40487.7233	2
11	29747	807.2585	2
12	29566	6893.2549	2
13	29890	9153.6054	2
14	30067	6895.41	2
15	29844	4216.0258	2
16	29596	2955.0542	2
17	29827	6434.0848	2

	CustomerID	1	2	3	4
1	11000	NULL	NULL	6344.8659	2770.2682
2	11001	NULL	650.8008	6403.3867	NULL
3	11002	NULL	NULL	8966.0143	NULL
4	11003	NULL	NULL	6319.4398	2674.4757
5	11004	NULL	NULL	6383.5298	2673.0613
6	11005	NULL	NULL	6339.6724	2634.3974
7	11006	NULL	NULL	6337.1309	2634.3974
8	11007	NULL	NULL	9073.1551	NULL
9	11008	NULL	NULL	8957.4726	NULL
10	11009	NULL	NULL	6267.8584	2673.0613
11	11010	NULL	NULL	6292.953	2644.3313
12	11011	NULL	NULL	8987.0093	NULL

Pivot

Create pivot table in SQL

SELECT

[non-pivoted column(s)], -- optional
[pivoted column(s)]

Data in rows

Pivoted data in columns
Values changed to columns

FROM

(SELECT query producing sql data for pivot) AS
TableAlias

Where to get the data from
Source select for the entire pivot
"input table"

PIVOT

(<aggregation function> (column for aggregation)
FOR [<column name containing values for pivot
table columns>]
IN ([first pivoted column], ..., [last pivoted column]
)
) AS PivotTableAlias

How to aggregate and what
value to display
As values in result

ORDER BY clause -- optional

Which data values should be
pivoted
Changed to columns

Pivot

Example:

SELECT

'AverageCost' AS Cost_By_Production_Days,
[0], [1], [2], [3], [4]

Data in rows

FROM

(SELECT ProductLine, DaysToManufacture,
StandardCost FROM Production.Product) AS
SourceTable

Data in cols

Where to get the data from

PIVOT (

AVG(StandardCost)

How to aggregate and
what value to display

FOR DaysToManufacture IN ([0], [1], [2],
[3], [4])

) AS PivotTable;

Where and what data
to pivot into columns

Pivot

```

SELECT
    'AverageCost' AS Cost_By_Production_Days,
    [0], [1], [2], [3], [4]

FROM
    ( SELECT ProductLine, DaysToManufacture,
      StandardCost FROM Production.Product ) AS
    SourceTable

PIVOT (
    AVG(StandardCost)
    FOR DaysToManufacture IN ([0], [1], [2],
    [3], [4])

) AS PivotTable;
    
```

	ProductLine	DaysToManufacture	StandardCost
207	NULL	1	0,00
208	NULL	0	0,00
209	NULL	0	0,00
210	R	1	1059,31
211	R	1	1059,31
212	S	0	13,0863
213	S	0	13,0863
214	M	0	3,3963
215	M	0	3,3963
216	S	0	13,0863
217	S	0	6,9223
218	S	0	38,4923
219	S	0	38,4923
220	S	0	38,4923
221	S	0	38,4923
222	R	1	868,6342
223	R	1	868,6342
224	R	1	868,6342
225	R	1	868,6342
226	R	1	868,6342
227	R	1	204,6251
228	R	1	204,6251
229	R	1	204,6251
230	R	1	187,1571
231	R	1	187,1571
232	R	1	187,1571
233	R	1	187,1571
234	R	1	187,1571
235	R	1	187,1571



	Cost_By_Production_Days	0	1	2	3	4
1	AverageCost	0,00	48,0699	NULL	NULL	NULL
2	AverageCost	15,599	238,9855	358,8025	NULL	933,77
3	AverageCost	7,9917	355,1017	360,9428	NULL	993,5262
4	AverageCost	24,5146	NULL	NULL	NULL	NULL
5	AverageCost	21,4237	266,1222	NULL	NULL	885,9342

Pivot

```

SELECT
    ProductLine,
    [0], [1], [2], [3], [4]

FROM
    ( SELECT ProductLine, DaysToManufacture,
      StandardCost FROM Production.Product ) AS
    SourceTable

PIVOT (
    AVG(StandardCost)
    FOR DaysToManufacture IN ([0], [1], [2],
    [3], [4])

) AS PivotTable;
  
```

	ProductLine	DaysToManufacture	StandardCost
207	NULL	1	0,00
208	NULL	0	0,00
209	NULL	0	0,00
210	R	1	1059,31
211	R	1	1059,31
212	S	0	13,0863
213	S	0	13,0863
214	M	0	3,3963
215	M	0	3,3963
216	S	0	13,0863
217	S	0	6,9223
218	S	0	38,4923
219	S	0	38,4923
220	S	0	38,4923
221	S	0	38,4923
222	R	1	868,6342
223	R	1	868,6342
224	R	1	868,6342
225	R	1	868,6342
226	R	1	868,6342
227	R	1	204,6251
228	R	1	204,6251
229	R	1	204,6251
230	R	1	187,1571
231	R	1	187,1571
232	R	1	187,1571
233	R	1	187,1571
234	R	1	187,1571
235	R	1	187,1571



	ProductLine	0	1	2	3	4
1	NULL	0,00	48,0699	NULL	NULL	NULL
2	M	15,599	238,9855	358,8025	NULL	933,77
3	R	7,9917	355,1017	360,9428	NULL	993,5262
4	S	24,5146	NULL	NULL	NULL	NULL
5	T	21,4237	266,1222	NULL	NULL	885,9342

Pivot

PIVOT is a powerful feature for reshaping data:

Ideal for data transformation for reporting and analysis

extremely useful when you need to summarize data across different categories and present it in a cross-tabular or matrix format

Potential Performance Optimizations

SQL Server database engine might be able to optimize PIVOT operations internally

PIVOT has weaknesses:

Quite complex

when dealing with very large datasets or when performing complex pivots with many columns and aggregations

Complexity for Dynamic Pivoting

all pivoted values need to be known in advance

Vendor Lock-in (T-SQL Specific)

other vendors require workarounds – like using CASE

Results		Messages	
	CustomerID	TotalDue	QR
1	29825	23153,2339	2
2	29672	1457,3288	2
3	29734	36865,8012	2
4	29994	32474,9324	2
5	29565	472,3108	2
6	29898	27510,4109	2
7	29580	16158,6961	2
8	30052	5694,8564	2
9	29974	6876,3649	2
10	29614	40487,7233	2
11	29747	807,2585	2
12	29566	6893,2549	2
13	29890	9153,6054	2
14	30067	6895,41	2
15	29844	4216,0258	2
16	29596	2955,0542	2
17	29827	6434,0848	2
...			-

Pivot – querying

PIVOT example:

```
SELECT
```

```
    CustomerID, TotalDue,
    DATEPART(quarter,
    OrderDate) AS QR
```

```
FROM
```

```
Sales.SalesOrderHead
er
```

Pivot – querying

	CustomerID	1	2	3	4
1	11000	NULL	NULL	6344.8659	2770.2682
2	11001	NULL	650.8008	6403.3867	NULL
3	11002	NULL	NULL	8966.0143	NULL
4	11003	NULL	NULL	6319.4398	2674.4757
5	11004	NULL	NULL	6383.5298	2673.0613
6	11005	NULL	NULL	6339.6724	2634.3974
7	11006	NULL	NULL	6337.1309	2634.3974
8	11007	NULL	NULL	9073.1551	NULL
9	11008	NULL	NULL	8957.4726	NULL
10	11009	NULL	NULL	6267.8584	2673.0613
11	11010	NULL	NULL	6292.953	2644.3313
12	11011	NULL	NULL	8987.0093	NULL
					

PIVOT example:

```
SELECT CustomerID, [1], [2], [3], [4]
FROM
    (SELECT CustomerID, TotalDue,
        DATEPART(quarter, OrderDate) AS QR
    FROM Sales.SalesOrderHeader
    ) AS SourceTable
PIVOT (
    Sum (TotalDue)
    FOR QR IN ( [1],[2],[3],[4] )
) AS PivotTable
```

Order by CustomerID

Pivot – querying

	Color	One	Two	Three	More
1	NULL	65,4935	NULL	NULL	3,6469
2	Black	527,042	649,3833	NULL	881,0484
3	Blue	631,4155	NULL	NULL	1078,407
4	Grey	NULL	NULL	NULL	125,00
5	Multi	NULL	NULL	NULL	59,865
6	Red	799,2041	NULL	NULL	1889,8871
7	Silver	503,2912	664,2133	NULL	1104,9095
8	Silver/Black	64,0185	NULL	NULL	NULL
9	White	NULL	NULL	NULL	9,245
10	Yellow	620,1584	594,83	NULL	1175,9272

PIVOT example:

```
SELECT * FROM(  
    SELECT Color, ListPrice,  
        CASE DaysToManufacture  
            WHEN 1 THEN 'One'  
            WHEN 2 THEN 'Two'  
            WHEN 3 THEN 'Three'  
            ELSE 'More'  
        END as [DTM]  
    FROM Production.Product  
    ) AS ProductInfo  
PIVOT (  
    AVG(ListPrice)  
    FOR [DTM] IN ( [One],[Two],[Three],[More] )  
    ) AS PivotTable  
ORDER BY Color
```

UnPivot

Carries out the opposite operation to PIVOT by rotating columns of a table-valued expression into column values.

Notice that UNPIVOT isn't the exact reverse of PIVOT.

PIVOT carries out an aggregation and merges possible multiple rows into a single row in the output.

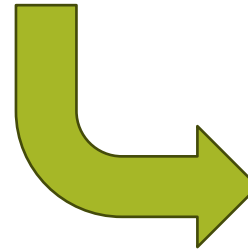
UNPIVOT doesn't reproduce the original table-valued expression result because rows have been merged.

Null values in the input of UNPIVOT disappear in the output.

When the values disappear, it shows that there may have been original null values in the input before the PIVOT operation.

UnPivot

	A	B	C	D	E	F
1	Respondent	Question 1	Question 2	Question 3	Question 4	Question 5
2	Ken Flerlage	1	4	4	5	2
3	John Smith	5	4	3	3	5
4	Willy Wonka	3	2	5	1	2
5	Fyodor Dostoevsky	5	2	2	1	
6	George Orwell	1	5	5	3	

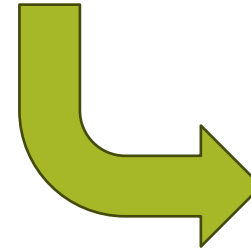


	A	B	C
1	Respondent	Question	Answer
2	Ken Flerlage	Question 1	1
3	Ken Flerlage	Question 2	4
4	Ken Flerlage	Question 3	4
5	Ken Flerlage	Question 4	5
6	Ken Flerlage	Question 5	2
7	John Smith	Question 1	5
8	John Smith	Question 2	4
9	John Smith	Question 3	3
10	John Smith	Question 4	3
11	John Smith	Question 5	5
12	Willy Wonka	Question 1	3
13	Willy Wonka	Question 2	2
14	Willy Wonka	Question 3	5
15	Willy Wonka	Question 4	1
16	Willy Wonka	Question 5	2

UnPivot

```
SELECT
    select_list
FROM table_name
UNPIVOT [INCLUDE | EXCLUDE
NULLS](
    unpivot_clause
    unpivot_for_clause
    unpivot_in_clause
);
```

	A	B	C	D	E	F
1	Respondent	Question 1	Question 2	Question 3	Question 4	Question 5
2	Ken Flerlage	1	4	4	5	2
3	John Smith	5	4	3	3	5
4	Willy Wonka	3	2	5	1	2
5	Fyodor Dostoevsky	5	2	2	1	
6	George Orwell	1	5	5	3	



	A	B	C
1	Respondent	Question	Answer
2	Ken Flerlage	Question 1	1
3	Ken Flerlage	Question 2	4
4	Ken Flerlage	Question 3	4
5	Ken Flerlage	Question 4	5
6	Ken Flerlage	Question 5	2
7	John Smith	Question 1	5
8	John Smith	Question 2	4
9	John Smith	Question 3	3
10	John Smith	Question 4	3
11	John Smith	Question 5	5
12	Willy Wonka	Question 1	3
13	Willy Wonka	Question 2	2
14	Willy Wonka	Question 3	5
15	Willy Wonka	Question 4	1
16	Willy Wonka	Question 5	2

UnPivot – querying

Results		Messages			
	Color	One	Two	Three	More
1	NULL	65,4935	NULL	NULL	3,6469
2	Black	527,042	649,3833	NULL	881,0484
3	Blue	631,4155	NULL	NULL	1078,407
4	Grey	NULL	NULL	NULL	125,00
5	Multi	NULL	NULL	NULL	59,865
6	Red	799,2041	NULL	NULL	1889,8871
7	Silver	503,2912	664,2133	NULL	1104,9095
8	Silver/Black	64,0185	NULL	NULL	NULL
9	White	NULL	NULL	NULL	9,245
10	Yellow	620,1584	594,83	NULL	1175,9272

```
SELECT
    Color, DTM, Value
FROM
    (SELECT Color, One, Two,
    Three, More FROM pvt) p
UNPIVOT
    (Value FOR DTM IN (One,
    Two, Three, More) ) AS
unpvt;
```

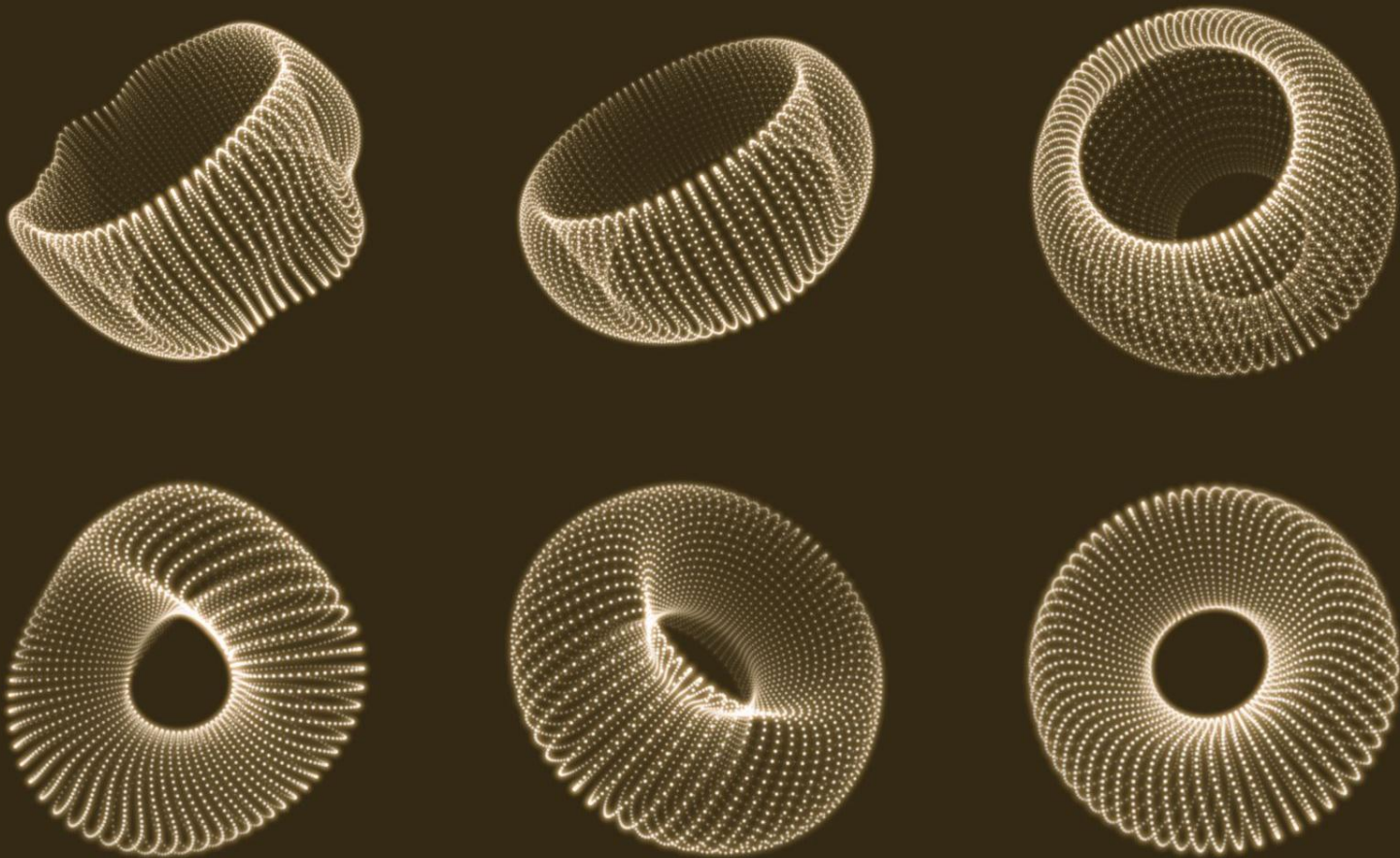

UnPivot – querying

	Color	One	Two	Three	More
1	NULL	65,4935	NULL	NULL	3,6469
2	Black	527,042	649,3833	NULL	881,0484
3	Blue	631,4155	NULL	NULL	1078,407
4	Grey	NULL	NULL	NULL	125,00
5	Multi	NULL	NULL	NULL	59,865
6	Red	799,2041	NULL	NULL	1889,8871
7	Silver	503,2912	664,2133	NULL	1104,9095
8	Silver/Black	64,0185	NULL	NULL	NULL
9	White	NULL	NULL	NULL	9,245
10	Yellow	620,1584	594,83	NULL	1175,9272



	Color	DTM	Value
1	NULL	One	65.4935
2	NULL	More	3.6469
3	Black	One	527.0420
4	Black	Two	649.3833
5	Black	More	881.0484
6	Blue	One	631.4155
7	Blue	More	1078.4070
8	Grey	More	125.0000
9	Multi	More	59.8650
10	Red	One	799.2041
11	Red	More	1889.8871
12	Silv...	One	503.2912
13	Silv...	Two	664.2133
14	Silv...	More	1104.9095
15	Silv...	One	64.0185
16	White	More	9.2450
17	Yell...	One	620.1584
18	Yell...	Two	594.8300

```
SELECT
    Color, DTM, Value
FROM
    (SELECT Color, One, Two, Three, More FROM pvt) p
UNPIVOT
    (Value FOR DTM IN (One, Two, Three, More) ) AS
    unpvt;
GO
```

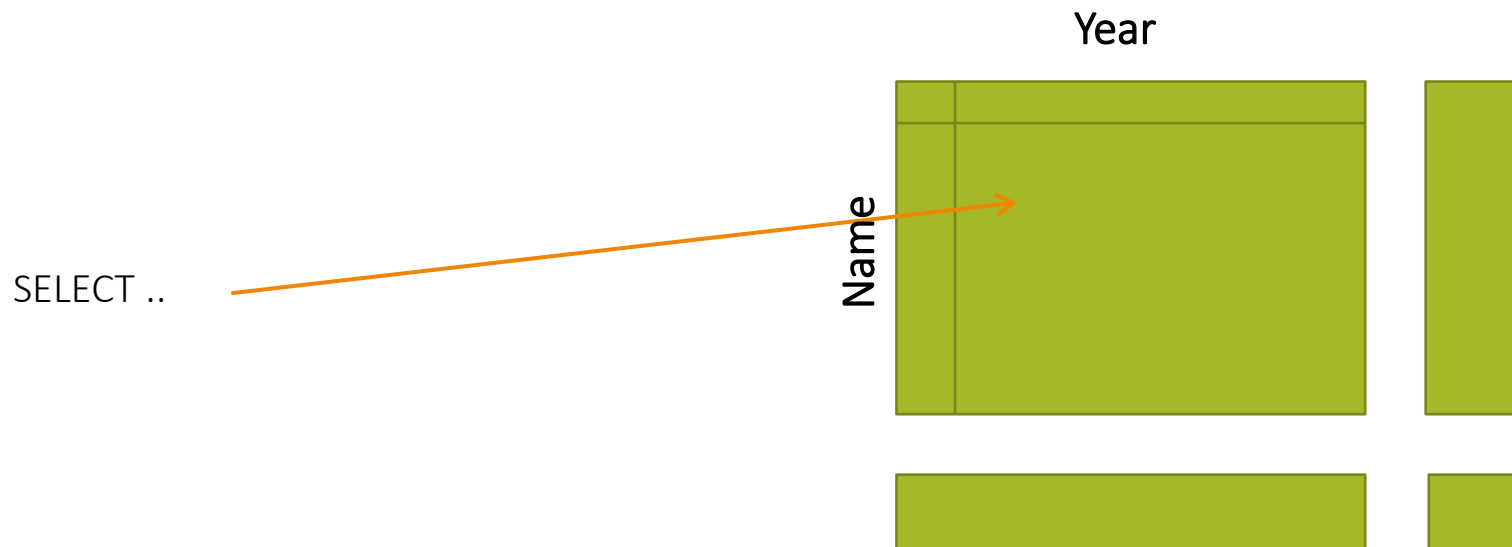


Grouping Sets

Grouping Sets

Multiple aggregations in SQL/92

Create a 2D spreadsheet that shows sum of columns and rows

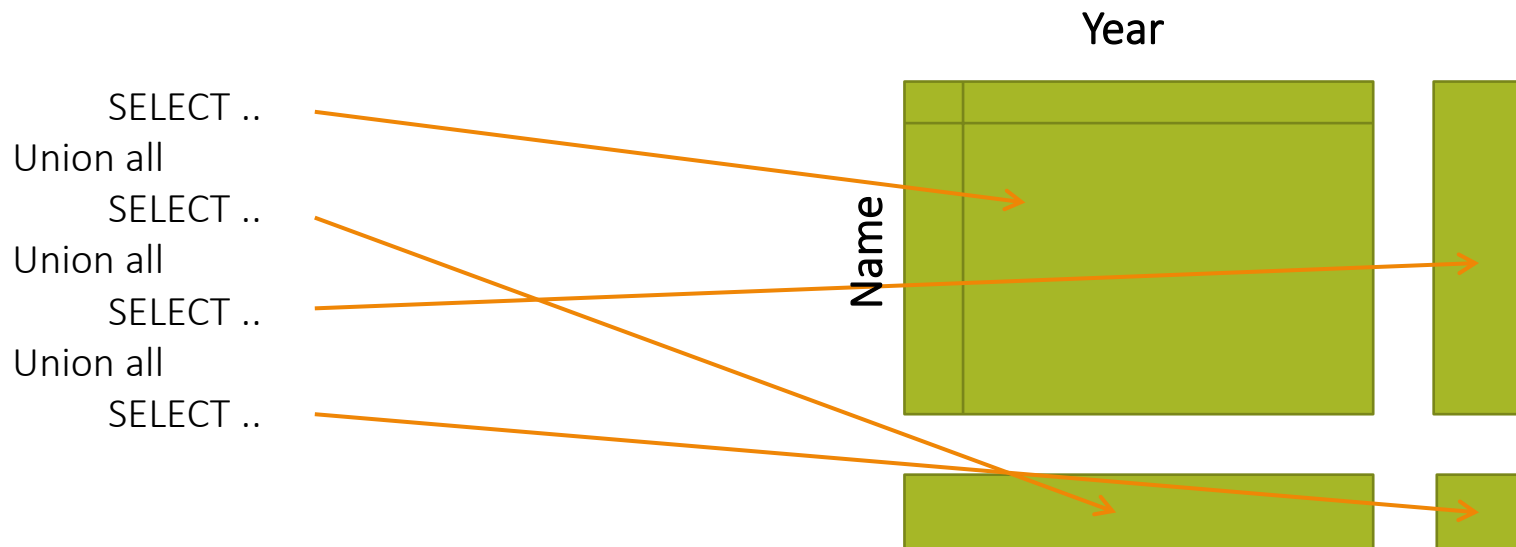


Grouping Sets

Multiple aggregations in SQL/92

Create a 2D spreadsheet that shows sum of columns and rows

each subtotal requires a separate aggregate query



Grouping Sets

Exemplar Star Schema

Student	ID	Name	Surname	Address
	S1	Bill	Inmon	Poznań
	S2	Ralph	Kimball	Piła
	S3	Bill	Gates	Poznań
	S4	Chris	Todman	Piła

Course	ID	Name	Type
	C1	Information Systems	Master
	C2	Software Engineering	Master

Students' grades	ID	Academic year	Student	Course	Professor	Grade
	01	D1	S1	C1	P1	3.5
	02	D1	S1	C1	P1	4.0
	03	D1	S1	C1	P2	5.0

Academic year	ID	Year	Semester
	D1	2003/04	Winter
	D2	2003/04	Summer
	D1	2004/05	Winter
	D2	2004/05	Summer

Professor	ID	Surname	Faculty
	P1	Słowiński	Computer Science
	P2	Stefanowski	Computer Science

Grouping Sets

Name	AVG(Grade)
Inmon	4.8
Kimball	4.7
Gates	4.0
Todman	4.5

SQL query – average grade by each professor:

```
SELECT Name, AVG(Grade)
FROM Students grades G,
Student S
WHERE G.Student = S.ID
GROUP BY Name;
```

Grouping Sets

SQL query – average grade by each professor and year:

```
SELECT Academic year, Name, AVG(Grade)
FROM Students grades G, Academic year A, Professor P
WHERE G.Professor = P.ID and G.Academic year = A.ID
GROUP BY Academic year, Name;
```

Academic_year	Name	AVG(Grade)
2001/2	Stefanowski	4.2
2002/3	Stefanowski	4.0
2003/4	Stefanowski	3.9
2001/2	Słowiński	4.1
2002/3	Słowiński	3.8
2003/4	Słowiński	3.6
2003/4	Dembczyński	4.8

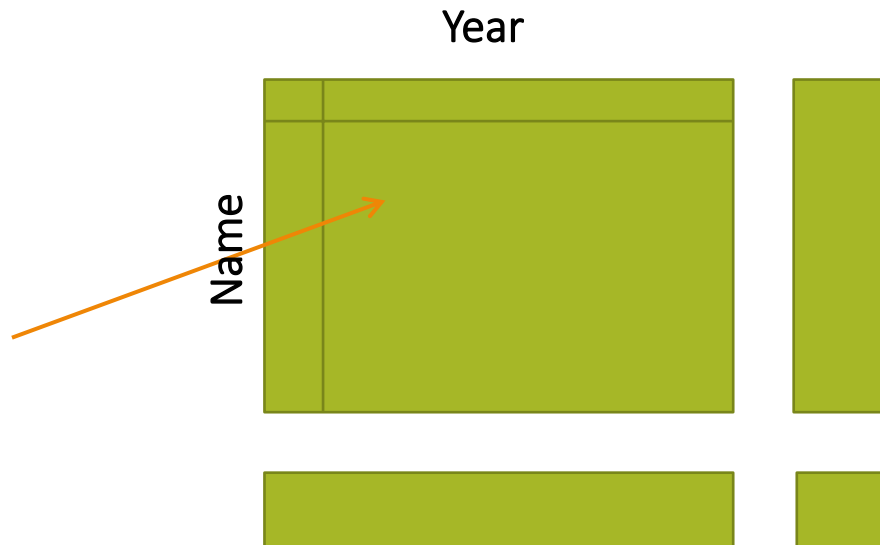
Grouping Sets

Multiple aggregations in SQL/92

Create a 2D spreadsheet that shows sum of columns and rows

each subtotal requires a separate aggregate query

```
SELECT Name, Year, avg(Grade)  
SUM  
FROM T GROUP BY Name, Year
```



Grouping Sets

Multiple aggregations in SQL/92

Create a 2D spreadsheet that shows sum of columns and rows

each subtotal requires a separate aggregate query

eg. sales by maker as well as car model

```
SELECT Name, Year, avg(Grade)
FROM T GROUP BY Name, Year
```

Union all

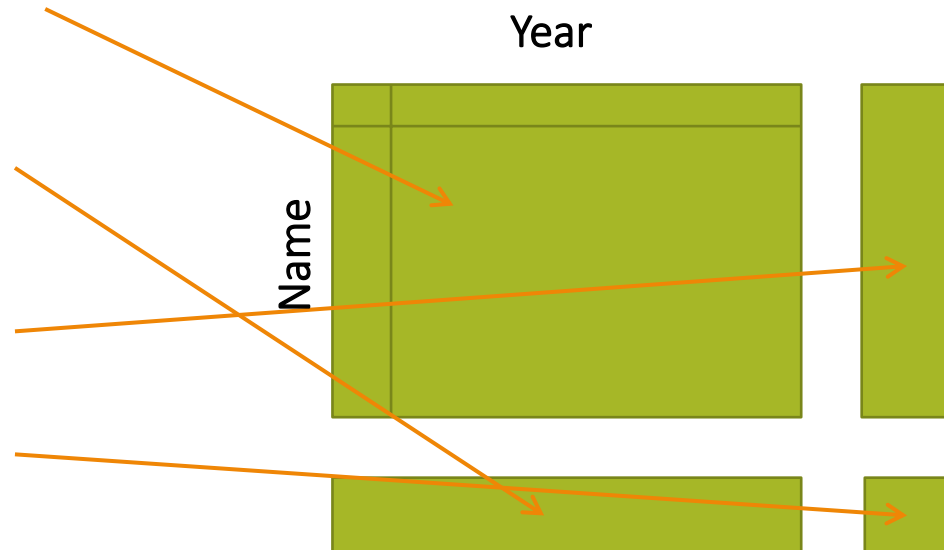
```
SELECT Name, '*', avg(Grade)
FROM T GROUP BY Name
```

Union all

```
SELECT '*', Year, avg(Grade)
FROM T GROUP BY Year
```

Union all

```
SELECT '*', '*', avg(Grade)
FROM sales
```



Grouping Sets

SQL3 extends SQL by:

Extensions to the GROUP BY operator

GROUP BY GROUPING SETS

*GROUPING SETS defines a list of sets of grouping columns and creates subtotals for **each** grouping union of GROUP BY*

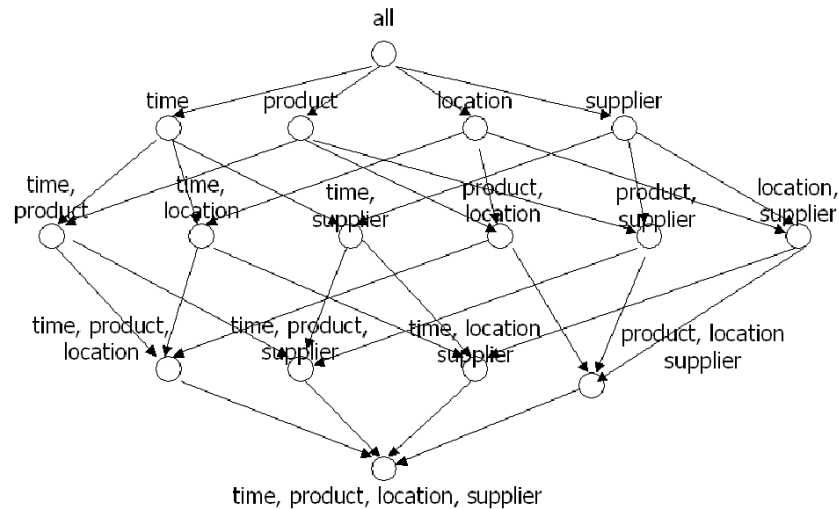
GROUP BY ROLLUP,

*ROLLUP enables a SELECT statement to calculate multiple levels of subtotals **across** a specified group of dimensions*

GROUP BY CUBE,

*CUBE takes a specified set of grouping columns and creates subtotals for **all** possible combinations of them*

Grouping Sets



Available groupings
TIME, PRODUCT,
LOCATION, SUPPLIER

Grouping Sets

GROUPING SET

Efficiently replaces a series of UNION-ed queries

The issue of NULL values

The new grouping functions generate NULL values at the subtotal levels

we have then generated NULLs and real NULLs from the data itself

How do we tell the difference?

Through the GROUPING function
GROUPING(.):

returns 0 for NULL in the data

returns 1 for generated NULL

Grouping Sets

GROUP BY GROUPING SETS

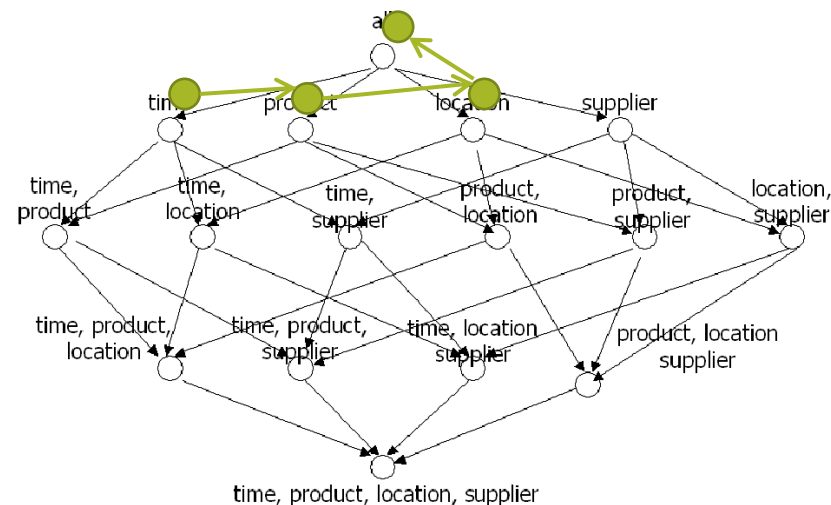
Explicitly state all required groupings

Comma separated list of groups - (dim1, dim2, ...)

SELECT Time, Product, Location, Supplier, SUM(Gain)

FROM Sales

GROUP BY GROUPING SETS ((Time), (Product), (Location), ());



Grouping Sets

GROUP BY GROUPING SETS

```
SELECT Time, "*", "*", "*", SUM(Gain)
```

```
FROM Sales
```

```
GROUP BY Time
```

UNION ALL

```
SELECT "*", Product, "*", "*",  
SUM(Gain)
```

```
FROM Sales
```

```
GROUP BY Product
```

UNION ALL

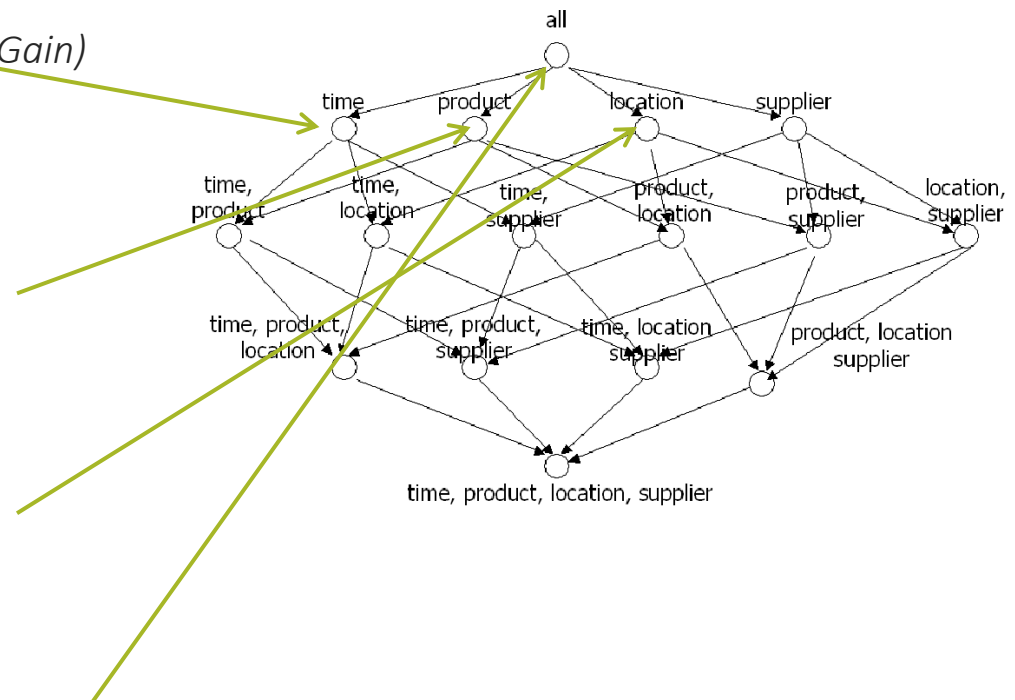
```
SELECT "*", "*", Location, "*",  
SUM(Gain)
```

```
FROM Sales
```

```
GROUP BY Location
```

UNION ALL

```
SELECT '*', '*', '*', '*', SUM(Gain)
```



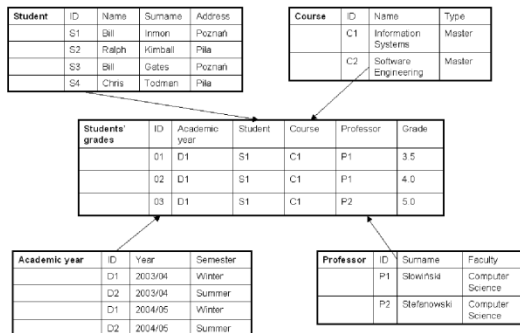
Grouping Sets

GROUP BY GROUPING SETS

SELECT Academic year, Name, AVG(Grade)

FROM Students grades

GROUP BY GROUPING SETS (Academic year, Name, ());



Academic_year	Name	AVG(Grade)
2001/2	NULL	4.15
2002/3	NULL	3.85
2003/4	NULL	3.8
NULL	Stefanowski	3.9
NULL	Słowiński	3.6
NULL	Dembczyński	4.8
NULL	NULL	3.95

Group by Grouping Sets

For such a query:

Select

P.name, PS.Name, PC.Name, AVG(P.ListPrice)

From

[Production].[Product] AS P

*inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID =
PS.ProductSubcategoryID*

*inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID*

Group By

grouping sets ((PC.name), (PS.name), (P.name))

What do we get in response?

	PSName	PCName	AVGPrice
1	Bib-Shorts	NULL	89.99
2	Bike Racks	NULL	120.00
3	Bike Stands	NULL	159.00
4	Bottles and Cages	NULL	7.99
5	Bottom Brackets	NULL	92.24
6	Brakes	NULL	106.50
7	Caps	NULL	8.99
8	Chains	NULL	20.24
9	Cleaners	NULL	7.95
10	Cranksets	NULL	278.99
11	Derailleurs	NULL	106.475
12	Fenders	NULL	21.98
13	Forks	NULL	184.40
14	Gloves	NULL	31.24
15	Handlebars	NULL	73.89
16	Headsets	NULL	87.0733
17	Helmets	NULL	34.99
18	Hydration Packs	NULL	54.99
19	Jerseys	NULL	51.99
20	Lights	NULL	31.3233
21	Locks	NULL	25.00
22	Mountain Bikes	NULL	1683.365
23	Mountain Frames	NULL	678.2535
24	Panniers	NULL	125.00
25	Pedals	NULL	64.0185
26	Pumps	NULL	22.49
27	Road Bikes	NULL	1597.45
28	Road Frames	NULL	780.0436
29	Saddles	NULL	39.6333
30	Shorts	NULL	64.2757
31	Socks	NULL	9.245
32	Tights	NULL	74.99
33	Tires and Tubes	NULL	19.4827
34	Touring Bikes	NULL	1425.2481
35	Touring Frames	NULL	631.4155
36	Vests	NULL	63.50
37	Wheels	NULL	220.9292
38	NULL	Accessori...	34.3489
39	NULL	Bikes	1586.737
40	NULL	Clothing	50.9914
41	NULL	Components	469.8602

```

Select PS.Name PSName, PC.Name PCName,
AVG(P.ListPrice) AVGPrice
From [Production].[Product] AS P
inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID = PS.ProductSubcategoryID
inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID
Group By grouping sets ( (PC.name), (PS.name))

```

Group by Rollup

ROLL-UP

produces a result set that contains
subtotal rows

regularly grouped rows

ROLLUP gives an aggregate value by
using the order of columns in a
GROUP BY clause

*will remove one record from grouping at a
time*

The last one

Group by Rollup

ROLL-UP

GROUP BY ROLLUP (a, b, c) is
equivalent to

GROUP BY GROUPING SETS

(a, b, c), (a, b), (a), ()

N elements of the ROLLUP translate
to (N+1) grouping sets

Order is significant to ROLLUP!

GROUP BY ROLLUP (c, b, a)

is equivalent with grouping sets of (c, b, a),
(c, b), (c), ()

Grouping Sets

Year	Brand	SUM(qty)	
2008	Mercedes	250	} (year, brand)
2008	BMW	300	
2008	VW	450	
2008		1000	} (year)
2009	Mercedes	50	} (year, brand)
...	...	...	
2009		400	} (year)
		1400	} (ALL)

ROLL-UP example:

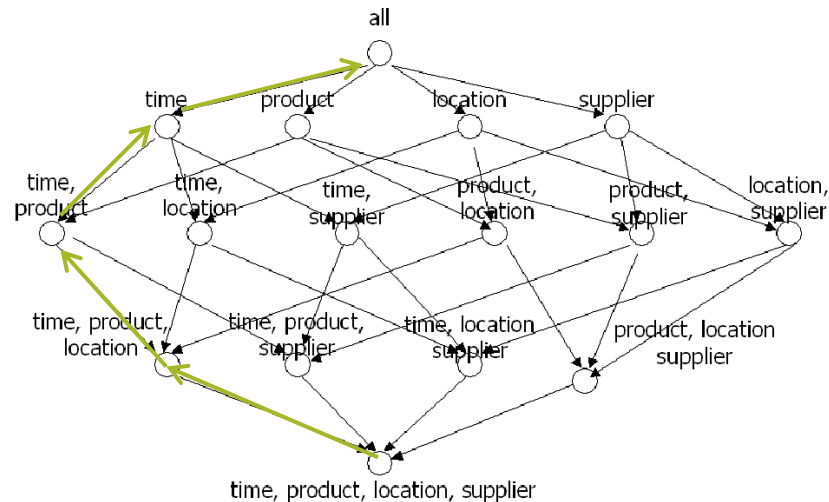
```
SELECT year, brand,  
SUM(qty) FROM sales  
GROUP BY  
ROLLUP(year, brand);
```

Grouping Sets

GROUP BY ROLLUP

Only hierarchical grouping is used

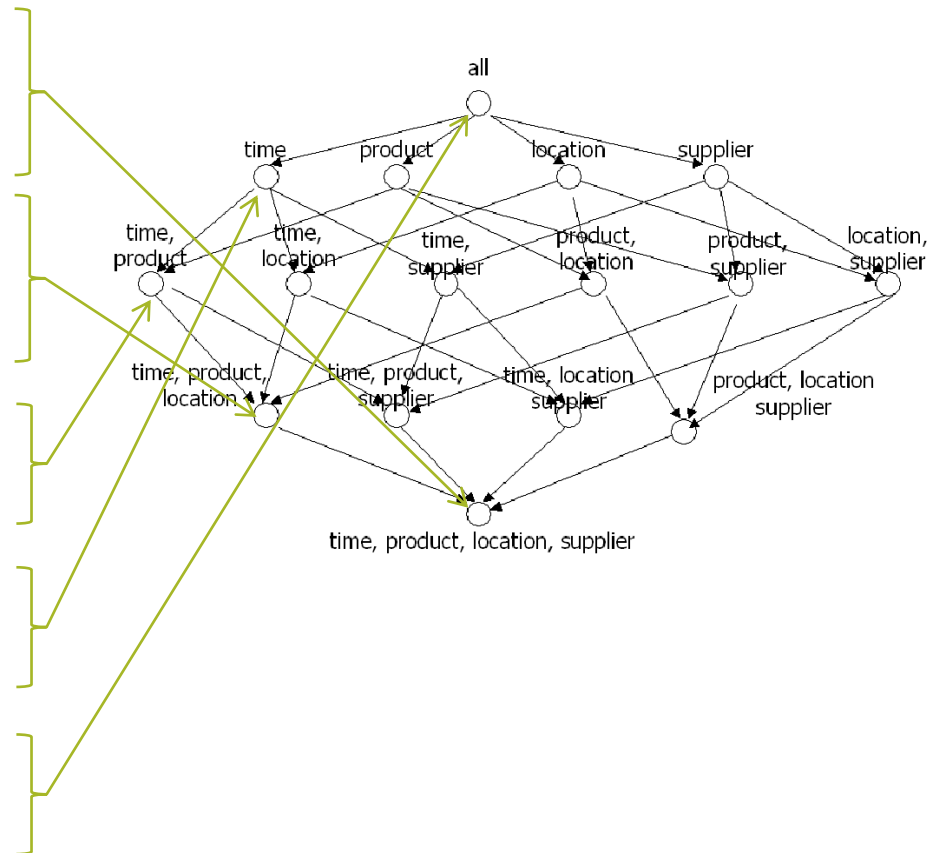
```
SELECT Time, Product, Location, Supplier, SUM(Gain) FROM Sales  
GROUP BY ROLLUP (Time, Product, Location, Supplier);
```



Grouping Sets

GROUP BY ROLLUP

```
SELECT Time, Product, Location, Supplier,  
SUM(Gain)  
FROM Sales  
GROUP BY Time, Product, Location, Supplier  
UNION ALL  
SELECT Time, Product, Location, "*", SUM(Gain)  
FROM Sales  
GROUP BY Time, Product, Location  
UNION ALL  
SELECT Time, Product, "*", "*", SUM(Gain)  
FROM Sales  
GROUP BY Time, Product  
UNION ALL  
SELECT Time, "*", "*", "*", SUM(Gain)  
FROM Sales  
GROUP BY Time  
UNION ALL  
SELECT '*, '*, '*, '*, SUM(Gain)  
FROM Sales;
```

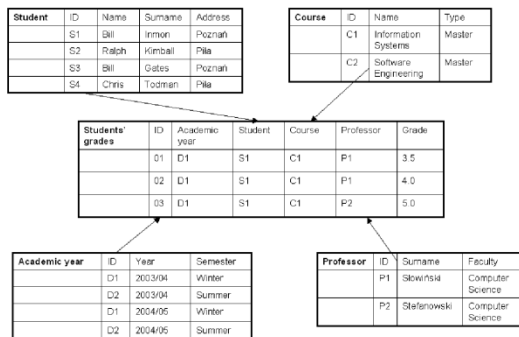


Grouping Sets

GROUP BY ROLLUP

SELECT Academic year, Name, AVG(Grade) FROM Students
grades G

GROUP BY ROLLUP(Academic year, Name);



Academic_year	Name	AVG(Grade)
2001/2	Stefanowski	4.2
2001/2	Słowiński	4.1
2002/3	Stefanowski	4.0
2002/3	Słowiński	3.8
2003/4	Stefanowski	3.9
2003/4	Słowiński	3.6
2003/4	Dembczyński	4.8
2001/2	NULL	4.15
2002/3	NULL	3.85
2003/4	NULL	3.8
NULL	NULL	3.95

Group by Rollup

For such a query:

Select

P.name, PS.Name, PC.Name, AVG(P.ListPrice)

From

[Production].[Product] AS P

*inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID =
PS.ProductSubcategoryID*

*inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID*

Group By

rollup (PC.name, PS.name, P.name)

What do we get in response?

	Name	Name	(No column name)
1	Bike Racks	Accessories	120.00
2	Bike Stands	Accessories	159.00
3	Bottles and Cages	Accessories	7.99
4	Cleaners	Accessories	7.95
5	Fenders	Accessories	21.98
6	Helmets	Accessories	34.99
7	Hydration Packs	Accessories	54.99
8	Lights	Accessories	31.3233
9	Locks	Accessories	25.00
10	Panniers	Accessories	125.00
11	Pumps	Accessories	22.49
12	Tires and Tubes	Accessories	19.4827
13	NULL	Accessories	34.3489
14	Mountain Bikes	Bikes	1683.365
15	Road Bikes	Bikes	1597.45
16	Touring Bikes	Bikes	1425.2481
17	NULL	Bikes	1586.737
18	Bib-Shorts	Clothing	89.99
19	Caps	Clothing	8.99
20	Gloves	Clothing	31.24
21	Jerseys	Clothing	51.99
22	Shorts	Clothing	64.2757
23	Socks	Clothing	9.245
24	Tights	Clothing	74.99
25	Vests	Clothing	63.50
26	NULL	Clothing	50.9914
27	Bottom Brackets	Components	92.24
28	Brakes	Components	106.50
29	Chains	Components	20.24
30	Cranksets	Components	278.99
31	Derailleurs	Components	106.475
32	Forks	Components	184.40
33	Handlebars	Components	73.89
34	Headsets	Components	87.0733
35	Mountain Frames	Components	678.2535
36	Pedals	Components	64.0185
37	Road Frames	Components	780.0436
38	Saddles	Components	39.6333
39	Touring Frames	Components	631.4155
40	Wheels	Components	220.9292
41	NULL	Components	469.8602
42	NULL	NULL	744.5952

```

Select PS.Name, PC.Name, AVG(P.ListPrice)
From [Production].[Product] AS P
inner join [Production].[ProductSubcategory] AS PS on
P.ProductSubcategoryID = PS.ProductSubcategoryID
inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID
Group By rollup ( PC.name, PS.name )

```

Group by Cube

CUBE operator:

- contains all the subtotal rows of a roll-up

- in addition contains all cross-tabulation rows

Can also be thought as a series of GROUPING SETs

All permutations of the cubed grouping expressions are computed along with the grand total

N elements of a CUBE translate to 2^n grouping sets

Grouping Sets

Year	Brand	SUM(qty)	
2008	Mercedes	250	} (year, brand)
2008	BMW	300	
2008	VW	450	
2008		1000	} (year)
2009	Mercedes	50	} (year, brand)
...	...	...	
2009		400	} (year)
	Mercedes	300	} (brand)
	BMW	350	
	VW	650	
		1400	} (ALL)

CUBE:

GROUP BY CUBE (a, b, c)

is equivalent to

GROUP BY GROUPING SETS(a, b, c) (a, b)
(a, c) (b, c) (a) (b) (c) ()

SELECT

year, brand, SUM(qty)

FROM sales

GROUP BY

CUBE (year, brand);

Grouping Sets

Year	Brand	SUM(qty)	
2008	Mercedes	250	} (year, brand)
2008	BMW	300	
2008	VW	450	
2008		1000	} (year)
2009	Mercedes	50	} (year, brand)
...	...	...	
2009		400	} (year)
	Mercedes	300	} (brand)
	BMW	350	
	VW	650	
		1400	} (ALL)

CUBE example:

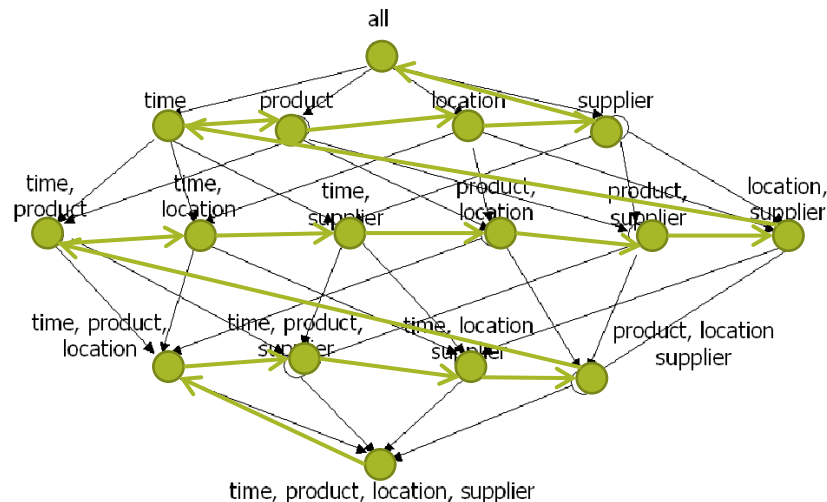
```
SELECT year, brand,  
SUM(qty) FROM sales  
GROUP BY CUBE (year,  
brand);
```

Grouping Sets

GROUP BY CUBE

All grouping combinations are used

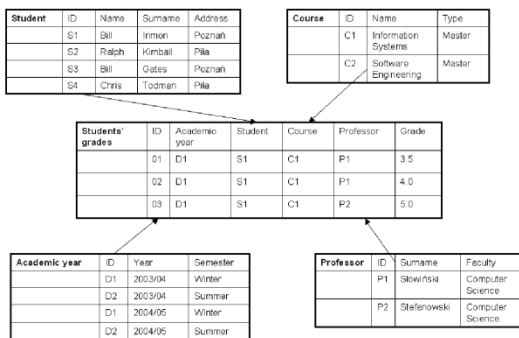
*SELECT Time, Product, Location, Supplier, SUM(Gain) FROM Sales
GROUP BY CUBE (Time, Product, Location, Supplier);*



Grouping Sets

GROUP BY CUBE

SELECT Academic year, Name, AVG(Grade) FROM Students grades
GROUP BY CUBE(Academic year, Name);



Academic_year	Name	AVG(Grade)
2001/2	Stefanowski	4.2
2001/2	Słowiński	4.1
2002/3	Stefanowski	4.0
2002/3	Słowiński	3.8
2003/4	Stefanowski	3.9
2003/4	Słowiński	3.6
2003/4	Dembczyński	4.8
2001/2	NULL	4.15
2002/3	NULL	3.85
2003/4	NULL	3.8
NULL	Stefanowski	3.9
NULL	Słowiński	3.6
NULL	Dembczyński	4.8
NULL	NULL	3.95

Group by Cube

For such a query:

Select

P.name, PS.Name, PC.Name, AVG(P.ListPrice)

From

[Production].[Product] AS P

*inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID =
PS.ProductSubcategoryID*

*inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID*

Group By

cube (PC.name, PS.Name, P.Name)

What do we get in response?

	name	Name	Name	(No column name)
1	All-Purpose Bike Stand	Bike Stands	Accessories	159,00
2	All-Purpose Bike Stand	Bike Stands	NULL	159,00
3	All-Purpose Bike Stand	NULL	NULL	159,00
4	AWC Logo Cap	Caps	Clothing	8,99
5	AWC Logo Cap	Caps	NULL	8,99
6	AWC Logo Cap	NULL	NULL	8,99
7	Bike Wash - Dissolver	Cleaners	Accessories	7,95
8	Bike Wash - Dissolver	Cleaners	NULL	7,95
9	Bike Wash - Dissolver	NULL	NULL	7,95
10	Cable Lock	Locks	Accessories	25,00
11	Cable Lock	Locks	NULL	25,00
12	Cable Lock	NULL	NULL	25,00
13	Chain	Chains	Components	20,24
14	Chain	Chains	NULL	20,24
15	Chain	NULL	NULL	20,24
16	Classic Vest, L	Vests	Clothing	63,50
17	Classic Vest, L	Vests	NULL	63,50
18	Classic Vest, L	NULL	NULL	63,50
19	Classic Vest, M	Vests	Clothing	63,50
20	Classic Vest, M	Vests	NULL	63,50
21	Classic Vest, M	NULL	NULL	63,50
22	Classic Vest, S	Vests	Clothing	63,50
23	Classic Vest, S	Vests	NULL	63,50
24	Classic Vest, S	NULL	NULL	63,50
25	Fender Set - Mountain	Fenders	Accessories	21,98
26	Fender Set - Mountain	Fenders	NULL	21,98
27	Fender Set - Mountain	NULL	NULL	21,98
28	Front Brakes	Brakes	Components	106,50
29	Front Brakes	Brakes	NULL	106,50
30	Front Brakes	NULL	NULL	106,50
31	Front Derailleur	Derailleurs	Components	91,49
32	Front Derailleur	Derailleurs	NULL	91,49

Select

P.name, PS.Name, PC.Name, AVG(P.ListPrice)

From

[Production].[Product] AS P

*inner join [Production].[ProductSubcategory] AS PS
on P.ProductSubcategoryID =
PS.ProductSubcategoryID*

*inner join [Production].[ProductCategory] AS PC on
PS.ProductCategoryID = PC.ProductCategoryID*

Group By

cube (PC.name, PS.Name, P.Name)

Case

CASE

Evaluates a list of conditions and returns one of multiple possible result expressions

similar to SWITCH in programming languages

Can be used in

statements such as SELECT, UPDATE, DELETE and SET

clauses such as IN, WHERE, ORDER BY and HAVING

Case

The CASE expression has two formats:

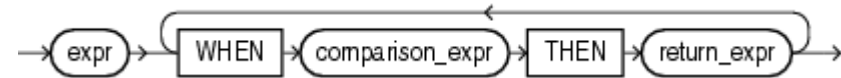
The simple CASE expression compares an expression to a set of simple expressions to determine the result.

CASE input_expression

WHEN when_expression THEN result_expression [...n]

[ELSE else_result_expression]

END



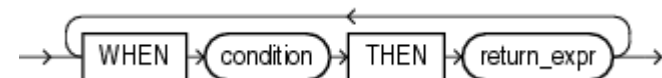
The searched CASE expression evaluates a set of Boolean expressions to determine the result.

CASE

WHEN Boolean_expression THEN
result_expression [...n]

[ELSE else_result_expression]

END



Case – querying data

	ProductNumber	Category	Name
1	AR-5381	Not for sale	Adjustable Race
2	BA-8327	Not for sale	Bearing Ball
3	BB-7421	Not for sale	LL Bottom Bracket
4	BB-8107	Not for sale	ML Bottom Bracket
5	BB-9108	Not for sale	HL Bottom Bracket
6	BC-M005	Mountain	Mountain Bottle Cage
7	BC-R205	Road	Road Bottle Cage
8	BE-2349	Not for sale	BB Ball Bearing
9	BE-2908	Not for sale	Headset Ball Bearings
10	BK-M18B-40	Mountain	Mountain-500 Black, 40
11	BK-M18B-42	Mountain	Mountain-500 Black, 42
12	BK-M18B-44	Mountain	Mountain-500 Black, 44
13	BK-M18B-48	Mountain	Mountain-500 Black, 48
14	BK-M18B-52	Mountain	Mountain-500 Black, 52
15	BK-M18S-40	Mountain	Mountain-500 Silver, 40
16	BK-M18S-42	Mountain	Mountain-500 Silver, 42

CASE example:

```
SELECT ProductNumber,  
       CASE ProductLine  
         WHEN 'R' THEN 'Road'  
         WHEN 'M' THEN 'Mountain'  
         WHEN 'T' THEN 'Touring'  
         WHEN 'S' THEN 'Other sale  
         items'  
         ELSE 'Not for sale'  
       END AS Category, Name  
FROM Production.Product  
ORDER BY ProductNumber;
```

Case – querying data

Results		
	Title	Gender
1	NULL	Unknown
2	Ms.	Female
3	Mr.	Male
4	Ms.	Female
5	Sr.	Unknown
6	Mrs.	Female

CASE example:

SELECT Title,

*CASE WHEN Title IN
('Ms.', 'Mrs.', 'Miss') THEN
'Female'*

WHEN Title = 'Mr.' THEN 'Male'

ELSE 'Unknown' END AS Gender

FROM Person.Person

Case – querying data

	ProductNumber	ProductName	Road	Mountain	Touring
482	BK-R19B-58	Road-750 Black, 58	X		
483	BK-T18U-44	Touring-3000 Blue, 44			X
484	BK-T18U-50	Touring-3000 Blue, 50			X
485	BK-M38S-38	Mountain-400-W Sil...		X	
486	BK-M38S-40	Mountain-400-W Sil...		X	
487	BK-M38S-42	Mountain-400-W Sil...		X	
488	BK-M38S-46	Mountain-400-W Sil...		X	
489	BK-M18S-40	Mountain-500 Silver...		X	
490	BK-M18S-42	Mountain-500 Silver...		X	

CASE example:

```
SELECT ProductNumber, Name AS  
ProductName,
```

```
    Road = CASE ProductLine WHEN  
            'R' THEN 'X' ELSE '' END,
```

```
    Mountain = CASE ProductLine  
                WHEN 'M' THEN 'X' ELSE '' END,
```

```
    Touring = CASE ProductLine  
               WHEN 'T' THEN 'X' ELSE '' END
```

```
FROM Production.Product
```

Case – querying data

CASE

SQL can produce a cross-tab via the CASE construct

Quite a complex query

All cases need to be known in advance

Example:

```
SELECT Store.state AS STATE,  
       SUM(CASE WHEN Time.quarter='Q1' THEN  
             sales END) AS Q1  
       ... ..  
       SUM(CASE WHEN Time.quarter='Q4' THEN  
             sales END) AS Q4  
FROM F, Store, Time  
WHERE F.storekey=Store.storekey AND  
       F.timekey=Time.timekey  
GROUP BY Store.state
```

Case – querying data

	CustomerID ▾	Q1 ▾	Q2 ▾	Q3 ▾	Q4 ▾
1	14324	5659.1783	NULL	NULL	NULL
2	22814	NULL	NULL	5.514	NULL
3	11407	NULL	NULL	NULL	59.659
4	28387	NULL	645.2869	NULL	NULL
5	19897	NULL	659.6408	NULL	NULL
6	15675	2580.1529	NULL	2699.9018	2682.9953
7	24165	3366.7583	NULL	NULL	NULL
8	27036	NULL	NULL	8.0444	NULL
9	18546	NULL	NULL	32.5754	NULL
10	11453	NULL	2633.1377	NULL	6402.4253
11	17195	2574.9042	NULL	1105.4834	NULL
12	17026	NULL	NULL	288.836	NULL
13	22768	NULL	NULL	663.5083	NULL

CASE example:

Select

CustomerID,

*SUM(CASE WHEN
DATEPART(quarter,OrderDate)=1 THEN
TotalDue END) AS Q1,*

*SUM(CASE WHEN
DATEPART(quarter,OrderDate)=2 THEN
TotalDue END) AS Q2,*

*SUM(CASE WHEN
DATEPART(quarter,OrderDate)=3 THEN
TotalDue END) AS Q3,*

*SUM(CASE WHEN
DATEPART(quarter,OrderDate)=4 THEN
TotalDue END) AS Q4*

from Sales.SalesOrderHeader

group by CustomerID

Case – querying data

	CustomerID ▾	Q1 ▾	Q2 ▾	Q3 ▾	Q4 ▾
1	14324	5659.1783	NULL	NULL	NULL
2	22814	NULL	NULL	5.514	NULL
3	11407	NULL	NULL	NULL	59.659
4	28387	NULL	645.2869	NULL	NULL
5	19897	NULL	659.6408	NULL	NULL
6	15675	2580.1529	NULL	2699.9018	2682.9953
7	24165	3366.7583	NULL	NULL	NULL
8	27036	NULL	NULL	8.0444	NULL
9	18546	NULL	NULL	32.5754	NULL
10	11453	NULL	2633.1377	NULL	3729.364
11	17195	2574.9042	NULL	1105.4834	NULL
12	17026	NULL	NULL	71.7919	NULL
13	22768	NULL	NULL	663.5083	NULL

CASE example:

Select

CustomerID,

*SUM(CASE WHEN
DATEPART(quarter,OrderDate)=1 THEN
TotalDue END) AS Q1,*

*AVG(CASE WHEN
DATEPART(quarter,OrderDate)=2 THEN
TotalDue END) AS Q2,*

*MIN(CASE WHEN
DATEPART(quarter,OrderDate)=3 THEN
TotalDue END) AS Q3,*

*MAX(CASE WHEN
DATEPART(quarter,OrderDate)=4 THEN
TotalDue END) AS Q4*

from Sales.SalesOrderHeader

group by CustomerID

Case – querying data

CASE example:

```
SELECT TotalDollar = SUM (SubTotal), TotalOrders = COUNT(*), PriceRange =  
    CASE  
        when SubTotal between 0 and 1000 then 'Small Order'  
        when SubTotal between 1000.0001 and 10000 then 'Average Order'  
        else 'Large Order'  
    END,  
FROM Sales.SalesOrderHeader  
GROUP BY  
    CASE  
        when SubTotal between 0 and 1000 then 'Small Order'  
        when SubTotal between 1000.0001 and 10000 then 'Average Order'  
        else 'Large Order' end  
ORDER BY TotalDollar desc
```

Results		Messages	
	PriceRange	TotalDollar	TotalOrders
1	Large Order	75224231,5819	1824
2	Average Order	31071401,9766	12577
3	Small Order	3550747,8454	17064



Common table expression

CTE

A Common Table Expression (CTE) is a temporary named result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.

WITH clause preceding the SELECT query allows you to name the relations created as a result of the subqueries placed there.

DMBS optimizes the execution of such a query, e.g. by implementing such a subquery in the form of a materialized view or a temporary table.

Subqueries defined within WITH can then be used in the query, making it more readable.

Suppose we want to find the sales territories that generated over \$5,000,000 in total revenue in a specific year. We can use a CTE to first calculate the totals, and then query the CTE to apply our filter.

-- Define the CTE query

```
WITH TerritoryYearlySales AS (  
    SELECT  
        t.[Name] AS TerritoryName,  
        YEAR(s.OrderDate) AS SalesYear,  
        SUM(s.TotalDue) AS TotalSales  
    FROM Sales.SalesOrderHeader s INNER JOIN  
        Sales.SalesTerritory t ON s.TerritoryID =  
        t.TerritoryID  
    GROUP BY t.[Name], YEAR(s.OrderDate)  
)
```

-- Consume the CTE

```
SELECT  
    TerritoryName, SalesYear, TotalSales  
FROM TerritoryYearlySales  
WHERE TotalSales > 5000000  
ORDER BY TotalSales DESC;
```

Example

CTE

Syntax:

```
[ WITH <common_table_expression> [ , ...n ] ]
```

```
<common_table_expression>::= expression_name [ ( column_name [ , ...n ] ) ] AS  
( CTE_query_definition )
```

expression_name

A valid identifier for the common table expression.

column_name

Specifies a column name in the common table expression.

The number of column names specified must match the number of columns in the result set of the CTE_query_definition.

The list of column names is optional only if distinct names for all resulting columns are supplied in the query definition.

CTE_query_definition

Specifies a SELECT statement whose result set populates the common table expression.

WITH

Common Table Expression (CTE)

Allows you to create a temporary result set that can be used in SELECT, INSERT, UPDATE, or DELETE queries.

Scope - CTEs are only available in the query in which they are defined and cannot be used in other queries.

Usage?

Readability

using CTE improves readability and makes it easier to maintain complex queries by breaking them down into simpler parts.

Recursion

One of the key features of CTEs is their ability to be recursive. Recursive CTEs are those that refer to themselves. This is particularly useful for hierarchical or tree-like data structures.

Preprocessing

CTEs can be used to organise data before it is finally processed, which can be useful in cases where data must be processed in a specific way before it can be used in the main query.

In data warehousing, CTEs are primarily used to break down complex, nested queries into readable, logical steps, much like creating variables in programming.

Why use them in DW?

- Readability: They replace messy nested subqueries.

- Reusability: You can reference the same CTE multiple times within the same execution scope.

- Recursion: They allow for recursive queries (e.g., querying organizational hierarchies).

```

WITH TerritorySalesBySalesperson AS (
    SELECT
        t.[Name] AS TerritoryName, p.FirstName + ' ' + p.LastName
        AS SalesPersonName, SUM(s.TotalDue) AS TotalSales
    FROM Sales.SalesOrderHeader s
        INNER JOIN Sales.SalesPerson sp ON s.SalesPersonID =
        sp.BusinessEntityID
        INNER JOIN Person.Person p ON sp.BusinessEntityID =
        p.BusinessEntityID
        INNER JOIN Sales.SalesTerritory t ON s.TerritoryID =
        t.TerritoryID
    GROUP BY
        t.[Name], p.FirstName, p.LastName
)

SELECT TerritoryName, SalesPersonName, TotalSales
FROM TerritorySalesBySalesperson
ORDER BY TerritoryName, SalesRank;

```

Example


```
-- CTE
```

```
WITH expression_name_1 (column_1,  
column_2,...,column_n) AS (
```

```
    CTE query definition 1
```

```
),
```

```
expression_name_2 (column_1,  
column_2,...,column_n) AS (
```

```
    CTE query definition 2
```

```
)
```

```
-- Final query using CTE
```

```
SELECT expression_A, expression_B, ...
```

```
FROM expression_name_2;
```

```
WITH SalesCount_CTE AS (  
    SELECT  
        SalesPersonID, TerritoryID, COUNT(SalesOrderID) AS  
        OrderCount  
    FROM Sales.SalesOrderHeader  
    WHERE SalesPersonID IS NOT NULL  
    GROUP BY SalesPersonID, TerritoryID  
) , FilteredTerritory_CTE AS (  
    SELECT *  
    FROM SalesCount_CTE  
    WHERE TerritoryID = 1 OR TerritoryID = 2  
)  
SELECT TerritoryID, AVG(OrderCount) AS  
AvgOrdersPerEmployee  
FROM FilteredTerritory_CTE  
GROUP BY TerritoryID;
```

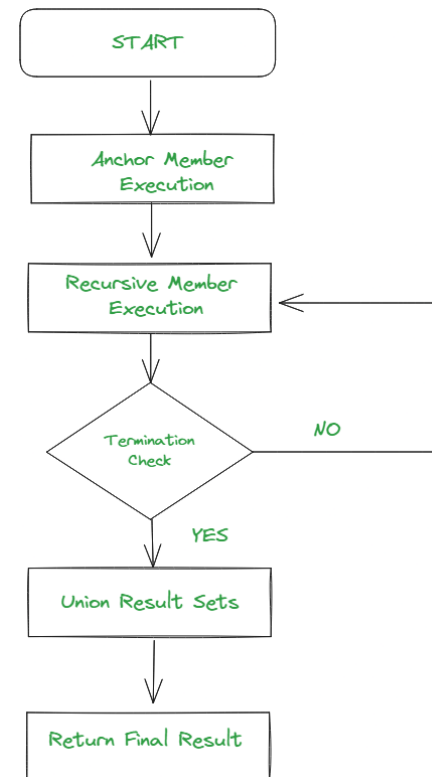
WITH

Recursive queries

Recursive CTEs are very useful when working with hierarchical or tree-like data structures, such as family trees, company organisational structures, or product categories.

Syntax:

```
WITH cte_alias (column_aliases) AS  
(  
    cte_query_definition --initialization  
    UNION ALL  
    cte_query_definition2 --recursive  
    execution  
)  
SELECT * FROM cte_alias
```



```
WITH DateGenerator AS (  
    -- Anchor: Start Date  
    SELECT CAST('2024-01-01' AS DATE) AS CalendarDate  
    UNION ALL  
    -- Recursive: Add one day until the limit is reached  
    SELECT DATEADD(DAY, 1, CalendarDate)  
    FROM DateGenerator  
    WHERE CalendarDate < '2024-12-31'  
)  
  
SELECT  
    CalendarDate, DATEPART(YEAR, CalendarDate) AS [Year],  
    DATEPART(MONTH, CalendarDate) AS [Month],  
    DATENAME(WEEKDAY, CalendarDate) AS [DayName]  
  
FROM DateGenerator  
  
OPTION (MAXRECURSION 366); -- Important: Default recursion  
limit is 100
```

Example

WITH

WITH Managers AS

(

--initialization

SELECT EmployeeID, LastName, ReportsTo

FROM Employees

WHERE ReportsTo IS NULL

UNION ALL

--recursive execution

SELECT e.employeeID, e.LastName, e.ReportsTo

FROM Employees e INNER JOIN Managers m

ON e.ReportsTo = m.employeeID

)

SELECT * FROM Managers

WITH

WITH Managers AS

```
(  
  --initialization  
    SELECT EmployeeID, LastName, ReportsTo  
    FROM Employees  
    WHERE ReportsTo IS NULL  
  UNION ALL  
  --recursive execution  
    SELECT e.employeeID, e.LastName, e.ReportsTo  
    FROM Employees e INNER JOIN Managers m  
    ON e.ReportsTo = m.employeeID  
)
```

SELECT * FROM Managers

OPTION (MAXRECURSION 4)

```

WITH CTE_Tree(employee_id, manager_id) AS (

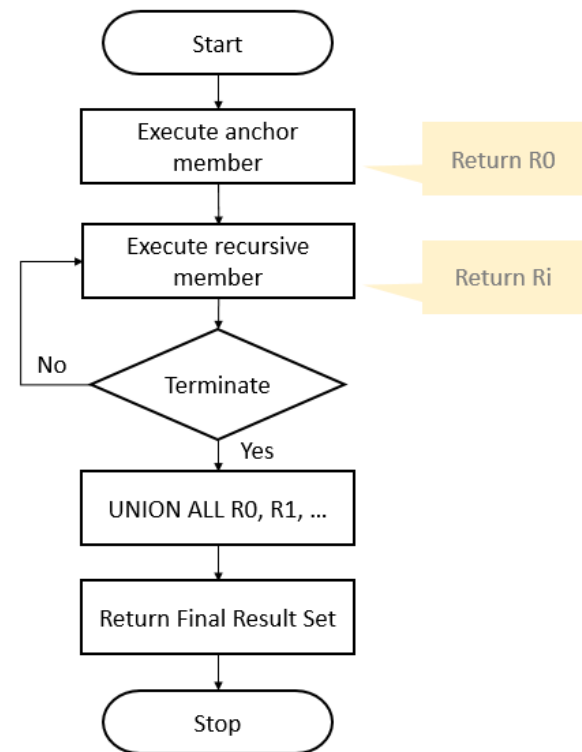
  -- Anchor member
  SELECT employee_id, manager_id FROM
  employees WHERE manager_id IS NULL

  UNION ALL

  -- Recursive member.
  SELECT t.employee_id, t.manager_id
  FROM employees t
  INNER JOIN CTE_Tree ct ON
  ct.employee_id = t.manager_id

)
SELECT * FROM CTE_Tree;

```



```
WITH cte_numbers(n, weekday)
AS (
    -- anchor
    SELECT 0, DATENAME(DW, 0)
    UNION ALL
    -- recursive
    SELECT n + 1, DATENAME(DW, n + 1)
    FROM cte_numbers
    WHERE n < 6
)
SELECT weekday
FROM cte_numbers;
```

weekday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

WITH

LVL + PATH

```
WITH CTE_Tree(employee_id, manager_id,  
lvl, path) AS (
```

```
    SELECT employee_id, manager_id, 1 as lvl,  
    TO_CHAR(employee_id)
```

```
    FROM employees WHERE manager_id IS NULL
```

```
    UNION ALL
```

```
    SELECT t.employee_id, t.manager_id, lvl+1, ct.path  
    || '-' || t.employee_id as path
```

```
    FROM employees t INNER JOIN CTE_Tree ct ON  
    ct.employee_id = t.manager_id
```

```
)
```

```
SELECT * FROM CTE_Tree;
```

EMPLOYEE_ID	MANAGER_ID	LVL	PATH
1	100	(null)	1 100
2	101	100	2 100-101
3	102	100	2 100-102
4	114	100	2 100-114
5	120	100	2 100-120
6	121	100	2 100-121
7	122	100	2 100-122
8	123	100	2 100-123
9	124	100	2 100-124
10	145	100	2 100-145
11	146	100	2 100-146
12	147	100	2 100-147
13	148	100	2 100-148
14	149	100	2 100-149
15	201	100	2 100-201
16	108	101	3 100-101-108
17	200	101	3 100-101-200
18	203	101	3 100-101-203
19	204	101	3 100-101-204
20	205	101	3 100-101-205
21	103	102	3 100-102-103

CTE

CTE can lead to infinite recursion – it is advisable to apply a restriction:

```
WITH CTE_Tree(employee_id, manager_id, lvl, path) AS (  
    SELECT employee_id, manager_id, 1 as lvl,  
    TO_CHAR(employee_id)  
        FROM employees WHERE manager_id IS NULL  
    UNION ALL  
    SELECT t.employee_id, t.manager_id, lvl+1, ct.path || '-' ||  
    t.employee_id as path  
        FROM employees t INNER JOIN CTE_Tree ct ON  
        ct.employee_id = t.manager_id  
        WHERE lvl < 10  
)  
SELECT * FROM CTE_Tree;
```

WITH

WITH cte_name (column_name [,...n])

AS (

CTE_query_definition -- Anchor member is defined.

UNION ALL

CTE_query_definition -- Recursive member is defined
referencing cte_name.

)

SELECT * FROM cte_name -- Statement using the CTE

[OPTION (MAXRECURSION X)] -- Limit recursion to X (query
hint)

With

Best practices – WITH Clause

Avoid nesting – use CTEs

Define CTEs in logical order

Explicitly name columns in CTEs

Avoid overly complex recursions

Document the purpose of each CTE

Test each CTE separately

	BigQuery 2025-10-01	Db2 (LUW) 12.1.1.2	MariaDB 12.0.2	MySQL 9.5.0	Oracle DB 23.26.0	PostgreSQL 18	SQL Server 2022	SQLite 3.50.0
With clause	✓ ^a	✓	✓	✓	✓	✓	✓	✓
With recursive clause	✓ ^b	✓ ^c	✓	✓	✓ ^d	✓	✓ ^d	✓
Over (...)	✓	✓	✓	✓	✓	✓	✓	✓

Rick Sherman,

Business Intelligence Guidebook: From Data Integration to Analytics, Elsevier Science, 2015

Martin Kleppmann

Designing Data-Intensive Applications, O'Reilly, 2017

David Loshin,

Business Intelligence: The Savvy Manager's Guide, Getting Onboard with Emerging IT, Morgan Kaufmann Publishers, 2003

Swain Scheps,

Business Intelligence for Dummies, Wiley Publishing, 2008

Efraim Turban, Ramesh Sharda, Dursun Delen, David King,

Business Intelligence (2nd Edition), Prentice Hall, 2010

Reza Rad,

Microsoft SQL Server 2014 Business Intelligence Development Beginner's Guide, Packt Publishing, 2014

Paul Weinberg and James Groff and Andrew Oppel

SQL The Complete Reference, Third Edition

McGraw-Hill, 2010

Beginning T-SQL 2012

Scott Shaw and Kathi Kellenberger

Apress, 2012

Bibliography

SOURCES